

TECHNICAL & EMPIRICAL REPORT

petriChip: The Silicon Petri Dish

A Comprehensive Technical & Empirical Report — Second Edition,
Rebuilt with Full Access to the Codebase and Raw Telemetry



Repository: github.com/titasmallick/petriChip

Contents

SECOND EDITION
FULL CODE + TELEMETRY ANALYSIS

1. Executive Summary

2. System Architecture

- 2.1 BionotesESP32.ino — Native Embedded Simulation
- 2.2 server.js — Node.js "NEAT-Lite" Architecture (the "Lite" Sandbox)
- 2.3 server_v2.js — The Biogeochemical Planetary Sandbox (the "Heavier" Sandbox)
- 2.4 server_v3.js — An Even Richer, but Far More Fragile, Sandbox
- 2.5 A Note on Version Naming

3. How the Neural Networks Work

- 3.1 The Reflex Arc
- 3.2 Biology-to-Code Mapping
- 3.3 Selection Without Gradients
- 3.4 The Mathematical Neuron, Worked by Hand
- 3.5 Forward Propagation Through a Full Layer
- 3.6 Gradient-Based Learning — the Road Not Taken
- 3.7 Evolutionary Mutation, Mathematically
- 3.8 Recurrent Memory
- 3.9 Motor Output: A Differential-Drive Controller
- 3.10 "NEAT Lite" vs. Standard NEAT
- 3.11 A Learning Path for Extending This from Scratch

4. Ecological & Biophysical Mechanics

- 4.1 Mass Conservation (Lindeman's Trophic Law)
- 4.2 Trophic Mutual Exclusivity
- 4.3 Allometric Scaling and r/K Selection
- 4.4 Climate, Seasonality, and Epidemiology (server_v2.js and server_v3.js only)
- 4.5 Key Design Differences Between the Three Web Builds

5. Empirical Findings

- 5.1 Heavier Sim (server_v2.js) — Full Observation Report
- 5.2 Lite Sim (server.js) — Full Observation Report, Including a 25-Hour Extension
- 5.3 Planetary Sandbox v3 (server_v3.js) — A Build That Never Gets Off the Ground

5.4 ESP32 Hardware (BionotesESP32.ino) — Real Hardware, a Stable Equilibrium, No Breakthroughs

5.5 Four-Way Comparative Analysis

6. Communicating the Project: Outreach Frameworks

7. Installation & Execution

7.1 ESP32 Microcontroller (BionotesESP32.ino)

7.2 Node.js Servers (server.js , server_v2.js , server_v3.js)

8. Known Limitations

8.1 Suggested Logging Improvements

9. Project Synthesis

1. Executive Summary

`petriChip` is an autonomous artificial-life (ALife) and neuro-evolutionary ecosystem framework. It reimagines the classic microbiology Petri dish as a computational medium — scaling from a \$5 ESP32 microcontroller up to a high-throughput Node.js simulation engine — to show how complex biological behavior emerges from simple physical and energetic rules rather than from any hand-written script.

There are no hardcoded behavior trees, no pre-trained networks, and no artificial loss function. Every behavior — foraging, wall avoidance, predator evasion, scavenging, mating, and speciation — is driven by an evolving neural network and body plan, shaped purely by survival and reproduction.

This second edition is built directly from the project's source code (all four implementations) and from full statistical analysis of every available telemetry log, rather than from documentation alone. That direct access changes several conclusions from the first edition, and adds an entire build — the planetary-sandbox successor, `server_v3.js` — plus a full hardware report for the ESP32 build that the first edition only described in passing. The headline results:

The heavier sandbox (`server_v2.js`) produced a genuine, hours-long evolutionary rescue.

One population, after 196 failed founder attempts, escaped the initial bottleneck and ran for 78.5 hours, reaching over 10,500 generations of lineage depth, sweeping to its speed cap, and settling into a dominant scavenger-based feeding strategy.

The lite sandbox (`server.js`) told a related but distinct story.

Its first long-surviving population lasted 16.8 hours before dying of a self-inflicted "poison flood," and a second, later population survived the same kind of flood by collapsing into a smaller, faster-turnover, permanently degraded state rather than dying outright — the first clear evidence in this project that self-poisoning does not have to be fatal.

The newest web build, `server_v3.js`, is a cautionary tale rather than a success story.

Across 1,010 logged extinctions and 20.8 hours of continuous runtime, not one population ever survived past a single generation of offspring. The most likely cause, confirmed by reading the code and modeling its terrain system, is a newly added aquatic/terrestrial mechanic so harshly tuned that most founders die from it before anything else in the simulation gets a chance to matter.

The real ESP32 hardware build tells a third, different story again.

Over 21.8 continuous hours it settled into a stable, high-turnover equilibrium and simply stayed there — no long-term drift in any trait, and no single population ever broke out into a long-lived lineage the way the two larger web sandboxes eventually did.

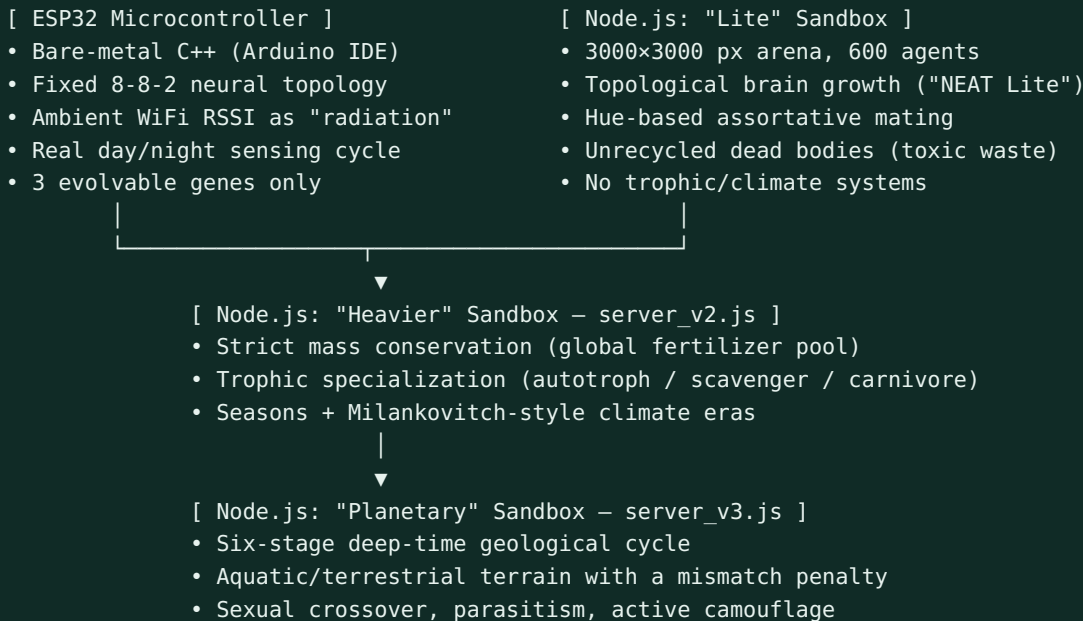
A single latent software bug

— a connection-evaluation order issue in the shared "NEAT Lite" brain code — is present in all three Node.js builds and confirmed by direct test. It means a large share of the "brain growth" visible in every log is neutral structural bloat rather than functional neural elaboration.

Read together, the four systems make a stronger and more honest point than any one of them alone: the same underlying rules (variation, heritability, differential survival) can produce a rich, self-sustaining evolutionary process, a self-poisoning collapse, a total and uniform failure to reproduce, or a stable non-evolving equilibrium — depending entirely on a handful of numerical constants and a couple of lines of un-executed code. That sensitivity is itself one of the project's most interesting findings.

2. System Architecture

`petriChip` is not one program but a lineage of four implementations of increasing — and, in one case, decreasing — biological and computational complexity. All four share the same core idea: agents with a body plan and a neural network live, eat, reproduce, and die inside a bounded arena with no external supervision.



2.1 `BionotesESP32.ino` — Native Embedded Simulation

The foundational build runs entirely on a single ESP32 microcontroller, hosting both the physics engine and the neural-network ecosystem in C++, with no external server.

- **Fixed-topology brain.** Each organism carries a compact 100-weight neural network: 8 sensory inputs, one hidden layer of 8 neurons, and 2 motor outputs (64 + 18 weights, padded to 100). Unlike every Node.js version, this topology never grows or shrinks — only the weights evolve.
- **Three evolvable genes.** `gene_speed`, `gene_vision`, and `gene_size` are the only heritable body-plan traits; there is no trophic, immunity, insulation, or aquatic system on this build.
- **A real day/night cycle.** A one-minute sine wave scales every creature's effective sensing radius between 50% and 100% of its base vision gene — a genuine circadian-style rhythm tied to the chip's own clock, not to any simulated calendar.
- **WiFi signal strength as environmental radiation.** Every 30 seconds the chip scans for nearby WiFi networks and scores how many have a strong signal (stronger than -60 dBm, roughly "a phone within a few metres"). That score becomes an environmental radiation value that speeds

up movement, adds positional jitter, raises metabolism, and occasionally seeds extra poison — a real physical-world input driving the simulated ecology.

- **Poison decay.** A small per-tick chance removes an existing poison item, a mitigation neither of the two lighter Node.js builds implements.
- **Edge dashboard.** A live view is served directly from the chip's own flash memory over a small HTTP/WebSocket server, viewable from any browser on the local network.

2.2 `server.js` — Node.js "NEAT-Lite" Architecture (the "Lite" Sandbox)

A server-side Node.js rewrite built for scale and open-ended brain growth, and the simplest of the three web builds.

- **Expanded arena.** A 3000×3000 coordinate space sustaining up to 600 concurrent organisms and 1,500 resource items, targeting roughly 200 simulation ticks per second (though see §5.2 and §5.4 for evidence that real achieved throughput is well below that nominal figure under load).
- **"NEAT Lite" brains.** Topology is no longer fixed — new nodes and connections can be added or removed by mutation, in principle letting cognitive capacity grow across generations. See §3.10 for why the "in principle" is doing a lot of work.
- **Cyclical short-term memory.** Organisms carry an internal memory vector that persists across ticks and feeds back into the next decision.
- **Hue-based assortative mating.** Reproduction requires phenotypic (colour) similarity, not any evolved "consent" signal.
- **No nutrient recycling.** Dead organisms leave behind permanent toxic corpse items with no decay path back into food — a design choice with major ecological consequences documented in §5.2.
- **Optional Gemini API observer.** Can periodically read the CSV telemetry log and generate natural-language hypotheses about the run, acting as an automated field biologist (not used to produce any of the analysis in this report, which is derived from the raw CSVs directly).

2.3 `server_v2.js` — The Biogeochemical Planetary Sandbox (the "Heavier" Sandbox)

The most ecologically complete of the three Node.js builds, adding strict thermodynamics and full trophic ecology on top of the lite architecture.

- **Closed nutrient cycle.** Every joule of metabolic burn, ageing cost, and corpse mass is returned to a single scalar, `globalFertilizer`. New plant food can only spawn if the soil pool can afford it — a hard mass-conservation law.
- **Trophic specialization.** Three mutually exclusive feeding genes — `gene_chloroplast` (autotrophy), `gene_scavenger` (decomposition), `gene_carnivore` (predation) — sum to at most 1.0, forcing an evolutionary trade-off.

- **Seasons and Milankovitch-style eras.** Four short seasons (spring/summer/autumn/winter) nest inside longer climate epochs (Ice Age, Greenhouse Drought, a normal "Holocene" state), each rescaling food spawn rates and thermal costs.
- **Thermal adaptation and epidemiology.** A `gene_insulation` trait offsets seasonal hot/cold penalties; winter also seeds viral outbreaks that spread on contact and drain energy unless countered by a `gene_immunity` trait.
- **Intent and kin signaling.** Dedicated neural outputs encode mating "consent" and aggression, introducing rudimentary kin-selection dynamics into reproduction and combat.

2.4 `server_v3.js` — An Even Richer, but Far More Fragile, Sandbox

The newest build examined in this report adds a striking amount of new biology on top of `server_v2.js` — most of which, as §5.3 documents in detail, never gets the chance to appear in the actual telemetry.

- **Deep-time geological epochs.** A six-stage cycle — Primordial Soup, Snowball Earth, Cambrian Bloom, Carboniferous ("High O₂"), The Great Dying (a Permian-style mass-extinction stage with toxic rain), and a modern "Cenozoic" stage — each with its own size cap (2.0 to 5.0) and climate multipliers, on a nominal 70,000-tick cycle.
- **Dynamic terrain.** A noise-based elevation map splits the arena into "land" and "water" regions. A `gene_aquatic` trait must match the local terrain or the creature suffers a heavy energy drain and a severe speed penalty — a real drowning/suffocation mechanic.
- **Sexual crossover.** Mating now runs a true gene-by-gene recombination between the two parents' neural networks, rather than simple weight-averaging.
- **True neural kin recognition.** Mate, predator, and symbiont decisions are driven by each creature's own evolved "intent" and "aggression" outputs, not by colour similarity as in the lite sandbox.
- **Parasitism.** A predator with a high `gene_parasite` value can, instead of eating its prey outright, "impregnate" it — storing a full copy of its own genome and brain on the victim.
- **Active camouflage.** A dedicated brain output lets a creature shift its own hue in real time.
- **Cordyceps-style host manipulation.** An infected creature's motor and aggression outputs are overridden by the simulation itself, forcing it to sprint toward and attack the nearest healthy creature — a genuine host-manipulation mechanic, not merely a passive infection tax.
- **Stricter, uncapped mass conservation.** Every unit of energy is accounted for through the fertilizer pool with no upper limit, compared to the heavier sandbox's earlier, capped version.

2.5 A Note on Version Naming

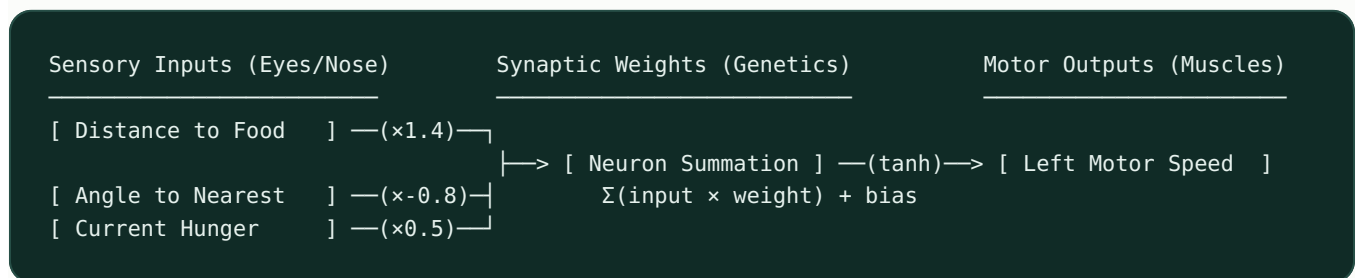
The original documentation for this project referred to a progression from `server_v2.js` through an eventual "`server_v4.js`," describing features such as dynamic terrain and parasitism as part of that later stage. Direct inspection of the actual source and logs available for this report shows those

features already present and running in `server_v3.js`, which is the build analyzed throughout §5.3. This report therefore treats `server_v3.js` as the current state of the planetary-sandbox line, and does not assume anything about a separate `server_v4.js` beyond what `server_v3.js` already contains.

3. How the Neural Networks Work

Commercial AI is trained top-down: massive labeled datasets, a loss function, and backpropagation. `petriChip` inverts this entirely — there is no supervisor and no gradient. Instead, cognition emerges through **differential reproductive success**: brains that happen to steer their owner toward food and away from poison survive to pass their weights on; brains that don't, die out.

3.1 The Reflex Arc



3.2 Biology-to-Code Mapping

Biological component	Neural equivalent	Code variable	Meaning
Sensory receptor	Input neuron	<code>inputs[0..7]</code>	Normalized proximity/angle to food, poison, kin, predators
Short-term memory	Recurrent vector	<code>nodes[8..15]</code>	Circular memory preserved across ticks
Synapse	Weight	<code>conn.w</code> (−5.0 to +5.0)	Excitatory or inhibitory connection strength
Axon hillock	Activation function	<code>Math.tanh(sum)</code>	Squashes summed input into (−1, 1)
Motor nerve	Output neuron	<code>newNodes[16]</code> , <code>newNodes[17]</code>	Differential left/right thruster output
Behavioral intent	Specialized outputs	<code>newNodes[18]</code> , <code>newNodes[19]</code>	Mating consent; aggression/bite reflex
Chromosome	Genotype	<code>c.brain.conns</code> + morphology genes	Inherited and mutated on reproduction

Each tick, an agent: (1) perceives distance/angle to nearby food, poison, kin and predators within its `gene_vision` radius; (2) propagates each input $\times$ weight into destination neurons; (3) squashes the sum through `tanh`; and (4) converts two of the output neurons into forward velocity and angular turn via a differential-drive controller — the same scheme a two-wheeled robot uses.

3.3 Selection Without Gradients

Reproduction and death are the only "training signal":

- **Fitness = survival.** An organism whose random weights steer it into poison loses energy and dies — its genes disappear from the population.
- **Energy thresholds** gate reproduction, via either costly asexual mitosis or (cheaper, but requiring a compatible, consenting partner) sexual recombination.
- **Mutation:** offspring inherit parental weights with a chance of small perturbations and a smaller chance of structural (topological) mutations.

The typical observed arc in a healthy run: at genesis, random weights mean most founders starve before finding food; within minutes, accidental positive food-seeking weight biases begin to survive; within an hour or two, coordinated chemotaxis (steering toward food, away from poison) becomes the norm across the population. §5 shows this arc actually playing out — and, in the case of `serv-er_v3.js`, almost never playing out at all.

3.4 The Mathematical Neuron, Worked by Hand

It's worth working through a single neuron before trusting the whole network. Take a neuron with three inputs — say, food detected (`x1`), poison detected (`x2`), and current energy (`x3`) — each with its own weight, plus a bias:

$$z = w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x3 + b$$

With `x = (0.8, 0.2, 0.6)`, `w = (1.5, -2.0, 0.5)`, `b = 0.1`:

$$\begin{aligned} z &= (1.5 \times 0.8) + (-2.0 \times 0.2) + (0.5 \times 0.6) + 0.1 \\ &= 1.2 - 0.4 + 0.3 + 0.1 \\ &= 1.2 \end{aligned}$$

The negative poison weight is doing real work here: as poison detection rises, it actively pulls the weighted sum down, independent of what food or energy are doing. That's the entire "decision," reduced to arithmetic — no understanding of "poison" is required, only a large negative number attached to that input.

Left alone, `z` is linear, and stacking any number of purely linear layers still collapses to one linear function — no amount of layering adds expressive power without a nonlinearity in between. This is why every weighted sum in `processBrain` is immediately passed through `tanh`:

Input to tanh	Output (≈)
-2	-0.964
-1	-0.762
0	0
1	0.762
2	0.964

```
const output = Math.tanh(z);
```

3.5 Forward Propagation Through a Full Layer

Generalizing to a layer of neurons, each neuron `j` computes its own weighted sum over all inputs `i`, then applies the activation:

```
zj = Σ(wji · xi) + bj
aj = f(zj)
```

Worked example — a hidden neuron with `x1=0.5`, `x2=0.8`, `w1=0.6`, `w2=-0.3`, `b=0.1`:

```
z = (0.5×0.6) + (0.8×-0.3) + 0.1 = 0.3 - 0.24 + 0.1 = 0.16
a = tanh(0.16) ≈ 0.159
```

This is exactly what `newNodes[c.out] += nodes[c.in] * c.w` followed by `Math.tanh(newNodes[i])` does across the whole node array — it's just this same operation repeated over every connection and every node, all at once.

3.6 Gradient-Based Learning — the Road Not Taken

It's worth being explicit about what `petriChip` does *not* do, since it's the default most people associate with "neural network": standard supervised training, via backpropagation.

That approach needs a **loss function** measuring how wrong a prediction is — commonly mean squared error, `L = (ŷ - y)²` — and then asks, for each weight, "if I nudge this weight slightly, how does the loss change?" That question is a derivative, `∂L/∂w`, and the **chain rule** is what makes it computable across layers:

$$dL/dw = dL/da \cdot da/dz \cdot dz/dw$$

For a `tanh` neuron specifically, `da/dz = 1 - a2` and `dz/dw = x`, so:

$$dL/dw = (dL/da) \cdot (1 - a^2) \cdot x$$

Once the gradient is known, **gradient descent** updates the weight against it:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot (\partial L / \partial w)$$

where `η` is the learning rate. A worked example: `w=0.5`, gradient `0.8`, `η=0.1` → `w_new = 0.5 - (0.1×0.8) = 0.42`. The classic toy problem for showing why any of this needs a hidden layer at all is **XOR** — its positive cases sit on opposite corners of the input space, so no single straight decision boundary (i.e. no single linear neuron) can separate them; a hidden layer with a nonlinearity is required.

None of this exists anywhere in `petriChip`. There is no loss function, no target output for a creature's motors to be "correct" against, and no gradient is ever computed or propagated. This section exists only to make the contrast in §3.7 concrete.

3.7 Evolutionary Mutation, Mathematically

What the codebase does instead, on every reproduction event:

$$w' = w + \varepsilon, \quad \text{where } \varepsilon \sim U(-s, s)$$

— a uniformly random perturbation of size up to `s` (`mutSev` in the code), with no reference to any error signal at all. Worked example: `w=0.7`, `s=0.1`, a drawn `ε=-0.06` → `w' = 0.7 - 0.06 = 0.64`.

	Backpropagation	Evolutionary mutation
Update source	Loss gradient	Random draw
Direction	Informed by the gradient	Undirected
Requires differentiability	Yes	No
Unit of optimization	One model, many updates	A population of candidates
Selection mechanism	None (built into the update rule)	Survival and reproduction filter the population afterward

Mutation alone is a weak search strategy; the reason it works here at all is that it's coupled to a brutal selection filter (the founder bottleneck documented throughout §5) that discards almost everything that doesn't work, generation after generation.

3.8 Recurrent Memory

Beyond the eight sensory inputs, each creature carries an 8-value memory array (`mem`) that is fed back into the network as if it were more sensory input:

```
for (let i = 0; i < 8; i++) { nodes[8 + i] = mem[i]; }  
// ...  
newMem: newNodes.slice(18, 26) // written back for next tick
```

Practically, this means two creatures standing in identical sensory conditions right now — same distance to food, same energy — can still act differently if their recent history differs, because that history is folded into the current computation as `newMem` from the previous tick. It's a simplified computational stand-in for the idea that a nervous system reacts to more than just its instantaneous inputs — not a biological model of memory, just a number that persists and feeds back.

3.9 Motor Output: A Differential-Drive Controller

The two motor outputs are converted into movement via a standard differential-drive scheme:

```
let v = (leftMotor + rightMotor) / 2.0;  
let omega = (rightMotor - leftMotor) / 10.0;  
c.angle += omega;  
c.x += Math.cos(c.angle) * v;  
c.y += Math.sin(c.angle) * v;
```

If both motors fire equally (`L=1, R=1`), `v=1` and `ω=0`: straight-line motion, no turning. If they fire in opposition (`L=1, R=-1`), `v=0` and `ω=-0.2`: the creature spins in place. The brain never touches position or angle directly — it only ever produces two numbers, and this fixed physical layer is what turns those numbers into motion, which in turn changes what the sensors report on the next tick. That sense→decide→act→sense loop is the entire behavioral engine, in all four implementations.

3.10 "NEAT Lite" vs. Standard NEAT

Worth flagging directly: the in-code label "NEAT Lite" borrows NEAT's central idea — brains that can add nodes and connections by mutation, rather than staying at a fixed topology — but it isn't a full implementation of the algorithm. It has no historical **innovation-number** bookkeeping (the mechanism standard NEAT uses to align matching genes between two parents during crossover), and, critically, its connection evaluation is a single pass over a partially-stale node array rather than a proper topological ordering.

This single-pass shortcut has a concrete consequence, and it is present in all three Node.js builds examined for this report (`server.js` , `server_v2.js` , and `server_v3.js`): a connection whose source is a hidden node, or another output node, reads a value of zero, because that source node has not yet been updated in the current pass. This was confirmed directly by constructing a minimal test brain in each codebase: a direct sensor-to-motor wire produces a real, non-zero motor signal, while the exact same wire split through one intermediate hidden node (as the "add node" mutation does) produces no signal at all.

With 16 possible input/memory sources and 12 possible output/memory destinations, only 192 genuinely functional direct connections can ever exist in any of these brains — everything routed through a hidden node is inert. This is the mechanism behind the "neutral genome bloat" finding that recurs throughout §5.1.13 and §5.2.5: a brain can accumulate hundreds of connections while its actual functional capacity stays essentially unchanged. It is a simplification made for speed, not an intentional design constraint.

3.11 A Learning Path for Extending This from Scratch

For anyone wanting to build the underlying neural-network machinery up by hand rather than treating it as a black box, a reasonable sequence:

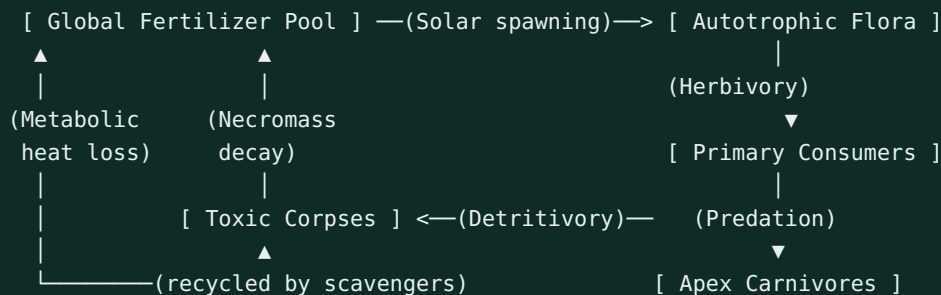
- 1. Fixed feedforward network** — a conventional layer-based net with clearly defined inputs, hidden neurons, and outputs, no evolution yet.
- 2. Manual verification** — print out weighted sums, activations and outputs for known inputs and check them by hand against §3.4–3.5.
- 3. Evolutionary optimization on a fixed topology** — mutate only weights, on a fixed brain shape (this is, in effect, what the ESP32 build already does), against a measurable fitness function.
- 4. Topology evolution done properly** — allow nodes/connections to be added or removed, this time with a correct evaluation order (fixing the §3.10 issue), so growth is functional, not just structural.
- 5. Full artificial-life experiments** — reintroduce sensors, energy, reproduction and selection pressure as controlled, individually testable variables.

A natural companion to that sequence is separating the current single-file `server.js` into modules — neural network code, evolutionary operators, and simulation/physics code kept apart, with basic unit tests on the neural math — not required to make the simulation run, but it makes each subsystem independently understandable and testable, and it would have caught the §3.10 issue immediately.

4. Ecological & Biophysical Mechanics

To prevent degenerate strategies (infinite energy loops, unbounded growth), the two richer sand-boxes enforce trade-offs grounded in real ecological and thermodynamic principles.

THE CLOSED-LOOP BIOGEOCHEMICAL ENGINE (server_v2 / server_v3)



4.1 Mass Conservation (Lindeman's Trophic Law)

In `server_v2.js` and `server_v3.js`, matter and energy are strictly conserved: every unit burned as metabolism or lost to ageing returns to `globalFertilizer`; a dead organism's surplus energy re-enters the soil while its mass becomes a toxic corpse item; and new plant food can only spawn if the soil pool can afford it. The practical result is Lindeman's efficiency law in miniature: a carnivore population can only be supported by a much larger underlying base of autotrophs and decomposers. The lite sandbox, `server.js`, deliberately does not implement this loop at all — a difference explored in depth in §5.2 and §5.5.

4.2 Trophic Mutual Exclusivity

$$\text{gene_chloroplast} + \text{gene_scavenger} + \text{gene_carnivore} \leq 1.0$$

- **Producers** (`chloroplast` > 0.5) photosynthesize free energy in spring/summer but suffer drag: $v \leftarrow v \times (1 - \text{chloroplast})$. Full autotrophs become effectively sessile.
- **Carnivores** (`carnivore` > 0.5) can damage and steal energy from prey, but plant food yield shrinks to $60 \times (1 - \text{carnivore})$ — plants become nearly useless to obligate carnivores.
- **Decomposers** (`scavenger` > 0.5) are the only organisms that can profit from corpses: carrion energy = $-80 + 140 \times \text{gene_scavenger}$, turning positive above roughly `scavenger` ≈ 0.57 .

4.3 Allometric Scaling and r/K Selection

In `server_v2.js` and `server_v3.js`, body size (`gene_size`, 0.5–2.5) sets an organism's life-history strategy via Kleiber-style scaling:

- Basal metabolic cost: $0.05 \times \text{size}^{0.75}$
- Kinetic (movement) cost: $\text{size} \times v^2 \times 0.01$
- **r-strategists** (small): cheap, fast-breeding, but weak in combat.
- **K-strategists** (large): expensive, slow-breeding, but dominant in contests and predation.

This makes larger bodies cheap per unit of mass at rest but expensive to move fast. The lite sandbox uses a different, purely linear cost structure with no such economy of scale — see §4.5.

4.4 Climate, Seasonality, and Epidemiology (`server_v2.js` and `server_v3.js` only)

Thermal cost penalizes cold exposure in winter ($(1 - \text{insulation}) \times 0.10$) and heat exhaustion in summer ($\text{insulation} \times 0.05$). Longer Milankovitch-style eras periodically shift the whole world into an Ice Age (food spawn halved, freezing cost doubled) or a Greenhouse Drought (heat exhaustion doubled). Winter also seeds viral outbreaks that spread on contact; $\text{viralTax} = \text{viralLoad} \times 0.5 \times (1 - \text{gene_immunity})$, so immunity trades a constant metabolic drain against infection resistance.

`server_v3.js` adds a second, slower cycle on top of this: a six-stage deep-time geological progression (§2.4) that is supposed to rescale size caps and radiation levels over tens of thousands of ticks. §5.3 shows that, in the run examined for this report, that cycle never actually advances past its first stage.

4.5 Key Design Differences Between the Three Web Builds

	<code>server.js</code> (lite)	<code>server_v2.js</code> (heavier)	<code>server_v3.js</code> (planetary)
Nutrient economy	Open — no recycling, corpses are permanent waste	Closed — strict mass conservation via <code>globalFertilizer</code>	Closed and uncapped
Mating trigger	Colour (hue) similarity	Mutual neural "intent"	Mutual neural "intent" plus size compatibility
Predation trigger	Size and energy edge only	Carnivore gene, aggression output, size edge	Same as v2, plus optional parasitism
Body-size cost curve	Linear in size, no economy of scale	Kleiber ($\text{size}^{0.75}$), economy of scale for large bodies	Same as v2
Terrain	None — a flat, uniform arena	None — a flat, uniform arena	Aquatic/terrestrial terrain with a mismatch penalty
Climate system	None	Seasons plus Ice Age/Greenhouse eras	Seasons, eras, and a (non-functional) deep-time cycle

This single structural progression — stripped-down, to closed-loop, to closed-loop-plus-terrain — turns out to be the single biggest driver of the very different outcomes documented in §5.

5. Empirical Findings

Across hundreds of hours of continuous, multi-generational logging on all four builds, detailed observation reports were compiled directly from the raw CSV telemetry. Every report below separates *what the data show* from *how it should be read*, with an explicit confidence label per finding, since every log in this project records only population means (no distributions, per-individual data, or event flags) — several readings are well-supported hypotheses, not proofs, and are labeled as such throughout.

5.1 Heavier Sim (`server_v2.js`) — Full Observation Report

Data: `evolution_v2_log.csv`, 680 rows, spanning four server sessions from 2026-09-17 11:14 UTC to 2026-09-21 03:07 UTC. Hours ("h") below are counted from the start of the surviving epoch (2026-09-17 20:37:24 UTC) unless stated otherwise.

5.1.1 Key findings

- 1. The world is a near-lethal filter.** 197 extinction-and-restart events were logged. Of 196 completed founder epochs, the median lasted **1.5 min**, 51% were over in under 1.5 minutes, and only 8 (4%) reached the 10-minute log, all of them on the brink (3–32 individuals).
- 2. One epoch escaped and ran for 78.5 hours** (3 d 6.5 h): **1.36 million births** $\approx$ **1.36 million deaths**, ancestry depth **10,516 generations**, still alive at the last row.
- 3. Evolutionary rescue.** Within ~10 minutes of genesis the survivors had roughly doubled their speed (1.75 $\rightarrow$ 3.62), more than tripled their brain wiring (3 $\rightarrow$ 11 connections), and begun scavenging (0.004 $\rightarrow$ 0.11).
- 4. Selection to a boundary.** Mean speed locked at **3.84 $\pm$ 0.04** against a hard cap of **4.0** and barely moved again (3.81 $\rightarrow$ 3.87 in ten-hour block means), so speed variation was effectively exhausted.
- 5. Detritivory became the dominant feeding strategy.** The scavenger gene averages ~0.7, while autotrophy (chloroplast) and predation (carnivore) hover at ~4–5%.
- 6. Directional thermal adaptation.** Insulation rose from 0.50 to ~0.85, as the code's cost structure predicts.
- 7. Two alternative body-size states** (mean $\approx$ 1.3 vs $\approx$ 2.2) that switch abruptly and persist for hours. The large state has longer lives, lower birth rate and higher standing population: an $r \rightarrow K$ style shift.
- 8. Punctuated dynamics.** Long stasis is broken by sudden population irruptions (peak **592 of a 600 cap** at 2026-09-19 13:37).
- 9. Seasonal nutrient cycle.** The soil pool is ~5.9 $\times$ larger in winter than spring, and population is ~21% lower in winter samples.

10. Neutral evolution is visible. Immunity random-walks across almost its whole range (0.06–0.97) because no epidemic ever took hold (never more than one infected creature).

11. Brain "complexity" grows linearly (~10 connections/hour) with no plateau, but this is probably mostly inert wiring: in the current code, hidden nodes never pass a signal on.

5.1.2 World mechanics that shape the biology

Mechanism in the code	Selective consequence
400 founders, random 1–5-wire brains, 1,000 energy each, 25% of initial items toxic	Harsh founder filter; strong early selection for foraging and against poison
Metabolism = $0.05 \cdot \text{size}^{0.75}$ (Kleiber) + $0.01 \cdot \text{size} \cdot v^2$ + vision + age + $0.0001 \times \text{connections}$	Larger bodies cheaper per unit mass; speed and brain size carry real costs
Plant item = $60 \cdot (1 - \text{carnivore})$; poison/carrion item = $-80 + 140 \cdot \text{scavenger}$	Scavenger gene is toxic-to-profitable switch at ~0.57; costs nothing on plants
$\text{chloroplast} + \text{scavenger} + \text{carnivore} \leq 1$; photosynthesis only in spring/summer; chloroplast makes movers sessile	Trophic trade-off; autotrophy is weak ($0.1 \cdot \text{gene}$ energy per tick)
Winter cost $(1 - \text{insulation}) \cdot 0.10$, summer cost $\text{insulation} \cdot 0.05$; Ice/Greenhouse eras rescale both	Normal-era optimum is insulation $\rightarrow 1$; Greenhouse era favours lower
Mitosis at energy > $300 \cdot \text{size}$ (child costs $200 \cdot \text{size}$); mating needs mutual intent and > $150 \cdot \text{size}$	Body size sets reproductive threshold, so it couples to life history
All burned energy and dead mass return to <code>globalFertilizer</code> ; food spawns in proportion to it	Closed nutrient cycle with seasonal spawn rate (spring 0.40, winter 0.05)
Winter viral seeding (50%), contact transmission, immunity reduces infection	Potential for host–pathogen selection (not realised)
Hard clamps: speed 0.5–4, size 0.5–2.5, vision 20–200	Directional selection can hit a wall

5.1.3 Chronology of the surviving run

Means over the ± 30 -minute window around each mark:

Time	Pop	Size	Speed	Connections	Scavenger	Chloroplast	Carnivore	Insulation	Immunity	Max lineage
Founders	600	1.50	1.75	3.0	0.004	0.004	0.004	0.50	0.50	1
10 min	296	1.22	3.62	11.0	0.11	0.080	0.057	0.35	0.54	76
h 1	181	1.67	3.85	18.8	0.43	0.058	0.051	0.63	0.36	222
h 2	162	1.64	3.79	29.7	0.76	0.064	0.050	0.76	0.21	335
h 10	210	0.97	3.82	123.5	0.71	0.073	0.056	0.87	0.35	1,72
h 20	189	2.08	3.81	233.3	0.63	0.062	0.048	0.90	0.87	3,13
h 30	190	1.22	3.86	331.9	0.62	0.045	0.039	0.91	0.43	4,55
h 40	153	1.18	3.89	435.8	0.79	0.048	0.054	0.84	0.32	5,98
h 50	197	1.87	3.85	497.6	0.57	0.053	0.038	0.87	0.69	7,21
h 60	275	2.30	3.87	604.1	0.78	0.050	0.035	0.85	0.38	8,55
h 70	241	2.28	3.85	665.2	0.79	0.056	0.036	0.80	0.62	9,71
h 77	207	2.33	3.87	716.6	0.69	0.044	0.035	0.79	0.84	10,4

Final row (2026-09-21 03:07): pop 527, 1,360,349 births, 1,360,622 deaths, max lineage 10,516, 730 connections.

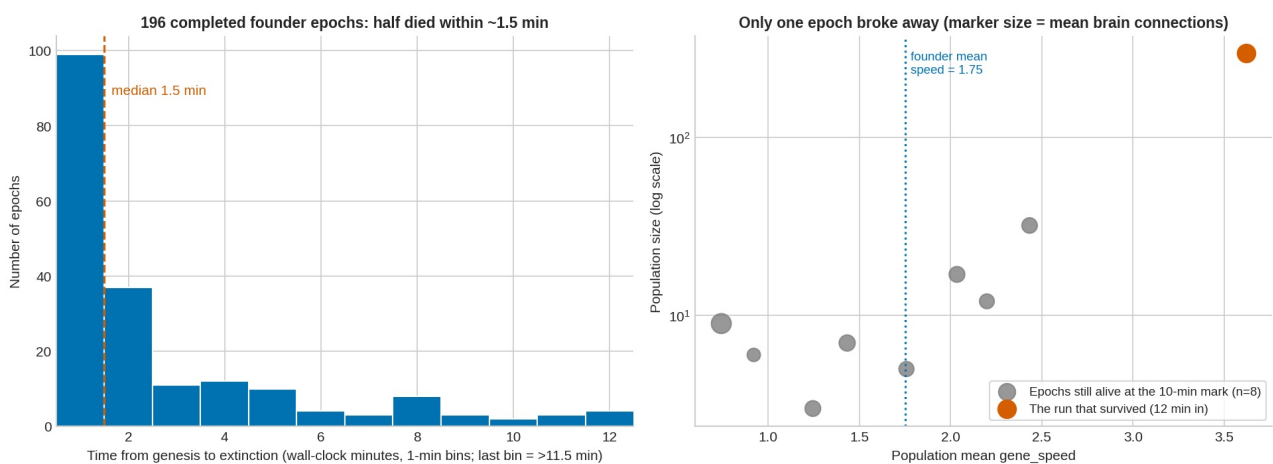


Figure 1. Left: the 196 completed founder epochs, heavily skewed toward immediate collapse. Right: the eight epochs still alive at the ten-minute mark versus the one epoch that went on to survive for 78.5 hours.

5.1.4 Founder bottleneck and repeated mass extinction

196 completed epochs (sessions 0–2) had median **1.5 min**, mean 2.8 min, interquartile range 1.1–3.5 min, longest 17.3 min; 99 epochs (51%) lasted under 1.5 minutes, only 8 (4%) lasted ≥ 10 minutes. Every genesis snapshot matched the random priors (mean size 1.502, speed 1.750, immunity 0.499,

insulation 0.499, 3.02 connections) — a sanity check that the log reflects the code. The 8 epochs that reached a timed log were all near-extinct (pop 3–32, median 8) and had already-larger brains (mean 7.5 vs 3.0 connections) than founders — read as **survivorship bias**: only creatures with working sensor-to-motor wiring endured. This is a classic **boom-and-bust founder crash**: founders expand toward the slot cap and then most lineages starve before selection can act. Confidence: high for the pattern, moderate for the mechanism.

5.1.5 Evolutionary rescue in the surviving epoch

At the first timed log (10 min post-genesis), population was 296 (from 600) and already far from the founder state: speed **3.62 (×2.1)**, connections **11 (×3.6)**, scavenger **0.11 (×28)**, chloroplast 0.08, carnivore 0.057, deepest lineage **76 generations**; the standing crop of poison items fell from **76 to 1**. Failed epochs at the same age had speed 0.74–2.43 and populations of 3–32. Reading: a population that would otherwise have gone extinct adapted fast enough (dozens of generations in minutes) to avoid it — one success in 197 tries can't separate a reproducible route from a lucky start. Confidence: high that rapid directional selection occurred, low on why this epoch and not others.

5.1.6 Directional selection to a genetic boundary (speed)

Mean speed was **3.84 ± 0.04** (SD across 471 windows), against a clamp of 4.0, exceeding 3.7 in 98.7% of windows and creeping up only slightly over 78 hours (3.81 → 3.87). Directional selection drove the trait into the hard clamp, after which variance is exhausted (fixation near the ceiling) — a constraint set by the model, not biology: the quadratic movement cost was clearly outweighed by foraging gains. Confidence: high.

5.1.7 Trophic evolution: scavenging dominates

The scavenger gene rose from 0.004 to ~0.43 by h 1 and ~0.76 by h 2, then stayed around **0.68 on average (range 0.28–0.87)**; chloroplast averaged ~0.05 and carnivore ~0.04, both drifting down slowly (the three sum to ~0.77, close to the ≤1 ceiling). After h 2 poison/corpse count was tiny (mean 1.8, max 8) while plant items averaged ~67–76 every season. Detritivory became the dominant niche because the gene turns a –80 toxin into up to +60 energy at zero cost on plants — close to a free advantage — and corpses are eaten almost as fast as they appear (a tight necromass recycling loop). No established autotroph or predator guild is visible, though a specialist subpopulation below ~4–5% can't be excluded from means alone. Confidence: moderate for the mechanism, high for the pattern.

5.1.8 Thermal adaptation: insulation

Insulation went from 0.50 to a mean of **0.845 (SD 0.075)** after h 1, above 0.8 in 81% of windows after h 5, with clusters of low values (< 0.72) around h 11.5–14.2, h 54.7–57, and shorter dips near h 66, 70 and 76. Under the code's costs the normal-era optimum is insulation = 1, so this is directional selection for cold tolerance held slightly below the cap by mutation; the low-insulation clusters are consistent with (unlogged) Greenhouse eras. Confidence: high for the trend, low for the era attribution.

5.1.9 Alternative stable states: body size and life history

Mean size is **bimodal** (modes ≈ 1.3 and ≈ 2.2) and switches abruptly:

State	Starts (UTC)	Approx. length
Small	from genesis (brief blip near h 2)	≈ 13.7 h
Large	09-18 10:27 (h 13.8)	11.6 h
Small	09-18 22:17 (h 25.7)	15.7 h
Large	09-19 14:07 (h 41.5)	7.5 h
Large (after brief dip)	09-19 22:27 (h 49.8)	3.5 h
Small	09-20 02:07 (h 53.5)	2.7 h
Large	09-20 04:57 (h 56.3)	22 h and continuing

Comparison after h 10:

	Small state	Large state
Mean size	1.29	2.16
Population	191	228
Births per minute	323	262
Mean lifespan (min)	0.62	0.90
Oldest individual (ticks)	11,100	16,500
Mean immunity	0.32	0.55
Soil nutrient pool	7.6k	12.8k

This is a **fast/slow (r vs K) life-history axis**: small bodies reproduce quickly and die young, large bodies breed slowly and live longer (deepest-lineage advance also slows from $\sim 170/h$ in h 1–10 to $\sim 105/h$ at the end, consistent with longer generation times). Size in the large state sits within $\sim 5\text{--}12\%$ of the 2.5 cap (a Cope's-rule-like drift limited by the clamp). Large bodies also carry the larger population — the opposite of Damuth's law — plausibly because the large state coincides with a richer nutrient pool. The small state reaches a mean of **0.88** at h 10.2, 41% below the founder mean, a resource-limited miniaturisation loosely echoing insular dwarfism. The abrupt switches with long stasis between them are read as **punctuated equilibrium** / clade replacement; means alone can't distinguish a whole-population shift from a change in clade mix. Confidence: high for the two states, moderate for the mechanism.

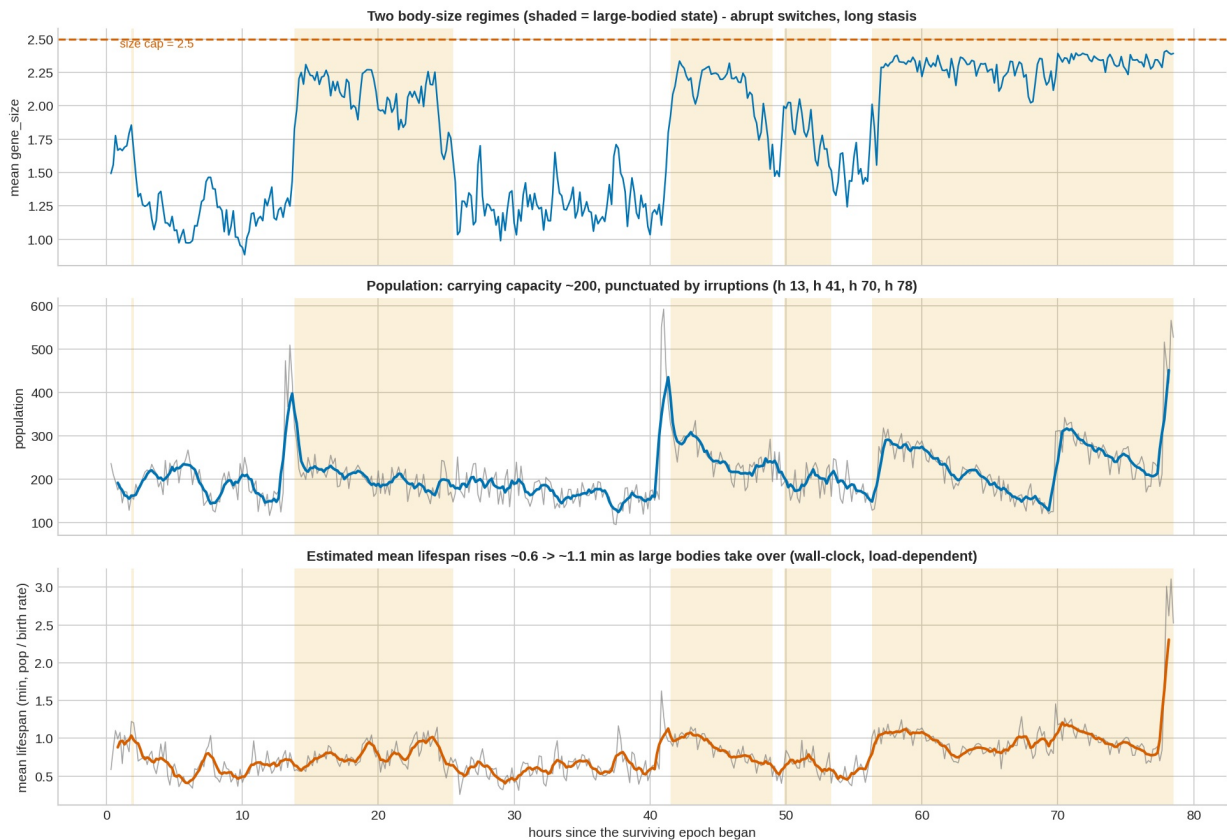


Figure 2. Top: mean body size switches abruptly between a small state and a large state (shaded), with long stasis in between and a hard ceiling at the 2.5 size cap. Middle: population, punctuated by sudden irruptions. Bottom: estimated mean lifespan.

5.1.10 Irruptions and sudden population upshifts

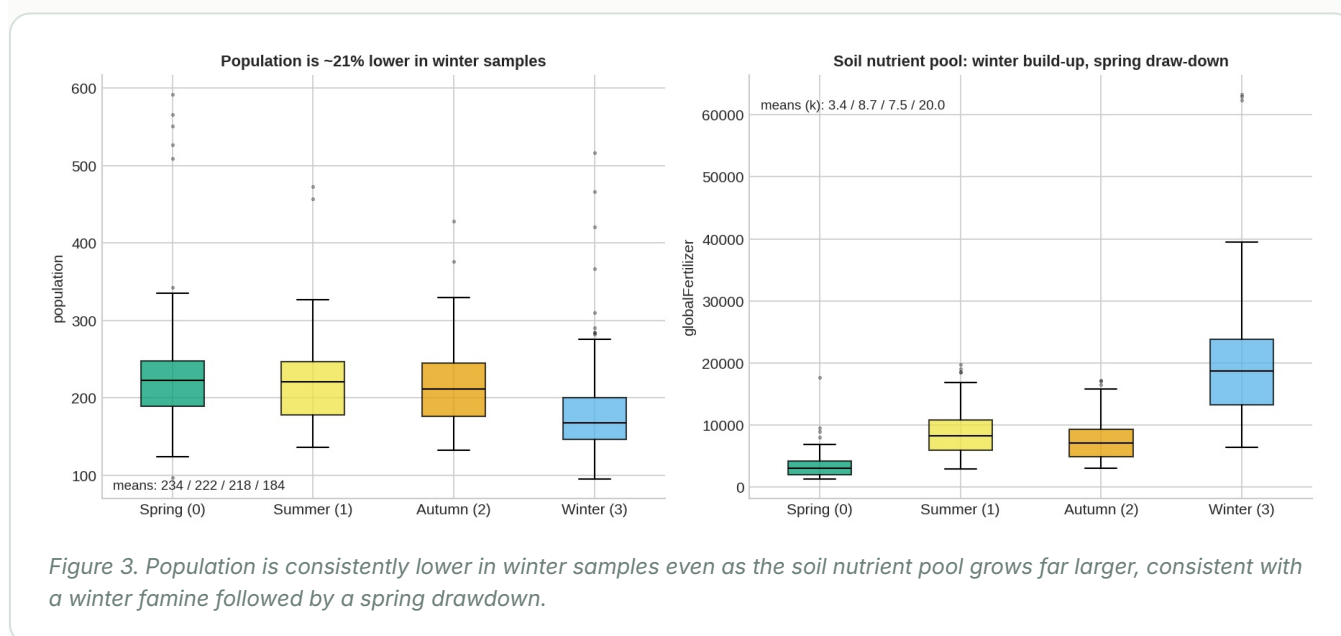
When (UTC)	Change	Notes
09-18 09:47 (h 13.2)	pop 168 → 473 , peak 509	Births $\approx 2\times$ baseline for ~ 1 h; large-body state takes over 40 min later
09-19 13:27 (h 40.8)	pop 153 → 551 , peak 592	98.7% of the 600 slot cap; back to ~ 275 in 1 h; large-body state takes over 40 min later
09-20 18:27 (h 69.8)	pop 125 → 310 , held 210–340 for ~ 5.5 h	No size change (already large)
09-21 02:27 (h 77.7)	pop 211 → 516 , max_age 17,170 → 7,229, fertilizer 26k → 63k	Only window where the oldest age fell by more than 45%

The first two read as classic irruption-then-overshoot-and-decay; the last looks like a mass mortality of old individuals followed by a recruitment pulse (severe era, manual meteor, or something else the log can't distinguish). The four upshifts show carrying capacity is not fixed — it stepped from ~ 200 to ~ 300 for hours. Confidence: high that shifts happened, low on causes.

5.1.11 Seasonal ecology and nutrient cycling

Season	Mean pop	Mean soil pool	Mean plant items
Spring	234	3.4k	73
Summer	222	8.7k	76
Autumn	218	7.5k	67
Winter	184	20.0k	73

Population differs significantly by season (Kruskal–Wallis $p \approx 3 \times 10^{-14}$) — a clean seasonal sawtooth: winter famine lets the soil pool build up as deaths return energy, and the spring bloom draws it back down ($\sim 5.9\times$ swing), with population $\sim 21\%$ lower in winter. Plant items stay flat across seasons (67–76) because spawns scale with $\text{pool} \times \text{rate}$ and the two move in opposite directions — a built-in nutrient buffer. Confidence: high for the sawtooth, moderate for the buffering reading.



5.1.12 Neutral evolution and the absence of epidemics (immunity)

Mean immunity wanders across **0.064–0.961** (mean 0.44, SD 0.25); 10-minute autocorrelation is 0.94, falling to 0.73 at 1 h and ~ 0.08 at 6 h; correlates with body size ($r = 0.35$). `infected_pop` was non-zero in only 14 of 473 windows, always exactly 1 creature, always in winter. The gene carries no cost when uninfected and infections never spread (effective $R < 1$), so there's no pathogen-driven selection — what's visible is genetic drift in a small population plus hitchhiking alongside the body-size sweeps. **The Red Queen hypothesis is not supported here.** Confidence: high for drift and no epidemics, moderate for hitchhiking.

5.1.13 Neural complexity growth (mostly neutral bloat)

Connections rose from 3 → 11 (10 min) → 124 (h 10) → 233 (h 20) → 436 (h 40) → 604 (h 60) → **730** (end), near-linear at ~10 connections/hour with no plateau — about 0.07 connections per generation against an expected +0.10 per birth from the mutation operator alone. At 730 connections the brain tax is ~76% of a size-2.4 body's basal cost. Growth at ~70%+ of what mutation alone would produce indicates selection is barely resisting — near-neutral accumulation under a weak drift barrier ("genome bloat"). Code check confirmed: connections read `nodes[c.in]` from the pre-propagation array, so any connection starting at a hidden or output node always contributes 0 (verified with a direct test: a sensor→motor wire gives a real signal, the same wire split through a hidden node gives none). Only 192 functional slots (16 inputs × 12 outputs) actually exist; add-node mutations functionally delete a live wire while adding a connection. Confidence: high for the growth, moderate-to-high for the inert-wiring reading.

5.1.14 Demographic steady state and longevity

Births and deaths agree to 0.02% over 1.36M events. After h 1, population averages **213 (SD 67, range 95–592)**, with ~17,300 births/hour. Estimated mean lifespan is 0.6–0.7 min in small-body periods and 0.8–1.0 min in large-body periods; record `max_age` is 26,858 ticks. The population sits at a resource-limited carrying capacity, not the 600 slot cap (approached only once); oldest individuals live several times the mean lifespan (a heavy longevity tail). Confidence: high.

5.1.15 Expected phenomena that were NOT observed

Host–pathogen cycles (Red Queen); clear photosynthetic or predatory guilds; predator–prey oscillations; speciation (not measurable — no hue/diversity/clade data logged); and long-run extinction (none in 78 h, unlike the first 196 epochs).

5.1.16 Summary table

Phenomenon	Evidence	Confidence
Founder bottleneck / mass extinction	196 epochs, median 1.5 min	High
Survivorship bias in near-extinct snapshots	8 epochs, brains 7.5 vs 3.0	High
Evolutionary rescue	Speed $\times 2.1$, connections $\times 3.6$ in 10 min	High (pattern) / Low (why)
Directional selection to a ceiling	Speed 3.84 ± 0.04 vs cap 4.0	High
Trophic specialisation (detritivory)	Scavenger ~ 0.7 ; poison ≈ 1	Moderate
Thermal adaptation	Insulation $0.50 \rightarrow 0.85$	High
$r \rightarrow K$ life-history states	Size, lifespan, birth rate contrast	Moderate–High
Punctuated equilibrium / clade replacement	Abrupt size switches	Moderate
Population irruption and overshoot	Peaks of 509 and 592	High (pattern) / Low (cause)
Carrying capacity	Pop ≈ 200 , births $\approx$ deaths	High
Seasonal cycle + nutrient cycling	Winter pool $\times 5.9$, pop -21%	High
Genetic drift / hitchhiking	Immunity $0.06\text{--}0.97$, no epidemics	High / Moderate
Neutral complexity accumulation	~ 10 connections/h, inert hidden nodes	Moderate–High
Red Queen	Not supported	n/a
Insular dwarfism	Loose analogy only	Low

5.2 Lite Sim (`server.js`) — Full Observation Report, Including a 25-Hour Extension

Data: `evolution_log.csv`, ultimately 504 rows across **46.8 continuous hours**, from 2026-09-21 03:45:35 UTC onward. This is a smaller, simpler cousin of the heavier sandbox: no seasons, no immunity/insulation genes, no trophic genes, and — critically — no nutrient-mass conservation at all. Mating is governed by colour (hue) similarity rather than a neural consent signal, and predation only requires a size and energy edge.

5.2.1 Key findings

- One lucky epoch ran for 16.8 hours**, then the world entered a **133-extinction, 4.8-hour mass-extinction spree** in which almost every fresh population died within about a minute.
- The long epoch swept to its speed cap and size floor within ~ 2 hours**, then held there (speed $3.6\text{--}3.9$ vs cap 4.0; size $0.5\text{--}0.7$ vs floor 0.5) — a fast, miniature body plan.

3. **The long epoch self-poisoned to death.** With no way to recycle corpses, poison items accumulated to as many as 888 of the 1,500 item slots while food was squeezed out; population drifted from ~90 down to ~45 before the eventual extinction.
4. **Brain wiring grew roughly linearly** for the whole 16.8 hours (~33 connections/hour, 3 → 458, no plateau) — and, as in the heavier sandbox, most of that wiring is functionally inert.
5. **A second long-surviving epoch (internally numbered "generation 190") later ran for 18.8 hours** — slightly longer than the first — and became the first observed case in this project of a population **surviving its own poison crisis rather than being wiped out by it** (see §5.2.6).
6. **A third population (generation 260) was still stabilizing** as the extended log ended, showing the same early trajectory the first two successes did.
7. **Population never approached the 600-slot capacity** across any of these attempts; the highest population ever recorded was 139.

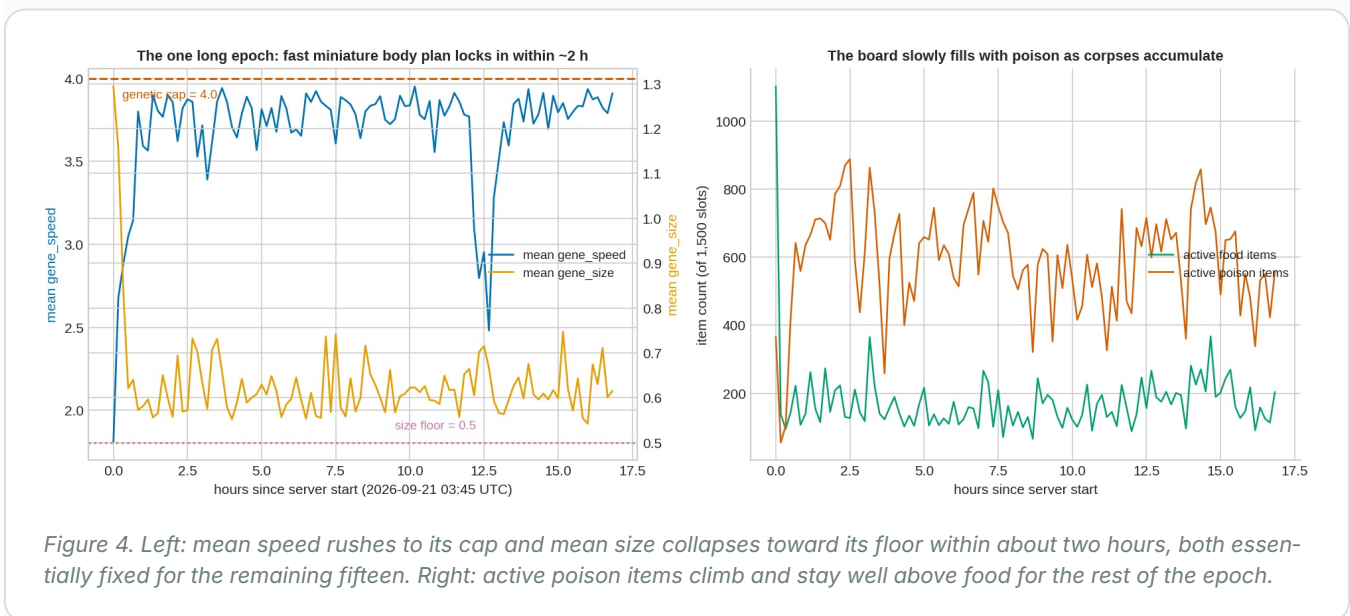
5.2.2 Mechanic differences from the heavier version

Mechanic	Lite (<code>server.js</code>)	Heavier (for contrast)
Founders	100 of 600 slots, energy 200 each, brains 1–5 wires	400 of 600 slots, energy 1000 each
Metabolism	$0.015 \cdot \text{size}$ baseline + $0.005 \cdot \text{size} \cdot (\text{motorL} + \text{motorR})$ — no allometric scaling	$0.05 \cdot \text{size}^{0.75} + 0.01 \cdot \text{size} \cdot v^2$ — cheaper per unit mass for large bodies
Poison	Fixed <code>-80</code> energy, no mitigating trait; dead bodies become permanent poison items	Offset by a <code>scavenger</code> gene; dead mass eventually decays back to a shared pool
Food/poison economy	No mass conservation — food spawns at a flat 20%/tick if a slot is free; poison only clears by being eaten	Closed nutrient cycle (<code>globalFertilizer</code>) with seasonal spawn rates
Mating trigger	Hue similarity (<code>hueDiff < 20</code>) plus energy/size thresholds — a phenotype-matching rule	Mutual neural "intent" output plus energy/size thresholds
Predation trigger	<code>size > 1.5×</code> defender and more energy; no trophic gene	<code>carnivore</code> gene + neural "aggression" output + <code>size > 1.2×</code>
Random death	Flat <code>0.00001</code> /tick chance	Age- and size-scaled senescence curve
Seasons/eras/radiation	None logged; only a manual radiation-burst trigger	Full season and era (Ice Age/Greenhouse) cycle

5.2.3 The first long epoch — rapid convergence to a fast, small body plan

Time	Pop	Mean size	Mean speed	Connections	Max lineage
Founders (t=0)	128	1.29	1.81	3.0	1
h 1	87	0.58	3.59	40.7	186
h 4	117	0.55	3.71	141.1	875
h 8	62	0.64	3.84	231.5	1,924
h 12	95	0.66	3.77	366.5	3,455
h 16.8 (last)	38	0.62	3.91	457.5	5,272

Mean speed exceeded 3.7 in 73% of windows; mean size bottomed at 0.542, barely above the 0.5 floor; size and speed are strongly anti-correlated ($r \approx -0.66$). This is directional selection to both boundaries at once: because movement cost here scales with motor output $\times$ size (not size $\times$ speed²) and baseline cost scales linearly with size (not size^{0.75}), there's no real energetic penalty for being small *and* fast. Confidence: high.



5.2.4 A world that poisons itself

Time	Food	Poison
t=0	1,102	366
h 4	189	669
h 8	105	545
h 12	88	435
h 16	217	480
last 20 rows, mean	194	588

Poison peaked at **888** active items (of 1,500 slots). Every death drops a permanent poison item with nothing to convert it back into food (no decay, no fertilizer, no scavenger gene); food spawns only into an empty slot, so as poison fills the arena, empty slots — and therefore new food — dwindle. The result is a slow self-poisoning feedback loop: more deaths → more poison → fewer empty slots → less food → more deaths. Population fell from a mean of ~87 in the first 4 hours to ~46 in the last 4 ($r \approx -0.66$ with hours elapsed). This mechanism, not predation or disease, is the most likely proximate driver of the eventual extinction at h 16.8. Confidence: moderate-high.

5.2.5 Brain growth, again mostly inert

Connections rose from 3 to 458 across the epoch, close to linear (~33/h average, ranging 14–41/h across 4-hour windows, $r \approx 0.99$ with elapsed time). As in the heavier sandbox, the same `process-Brain` single-pass evaluation issue applies: a connection sourced from a hidden node or another output always reads 0, confirmed here with the identical direct test. With only 192 possible functional direct slots, most of this near-perfectly-linear "brain growth" is very likely inert wiring accumulating under weak or no selection. Confidence: high for the growth pattern, moderate-to-high for the inert-wiring interpretation.

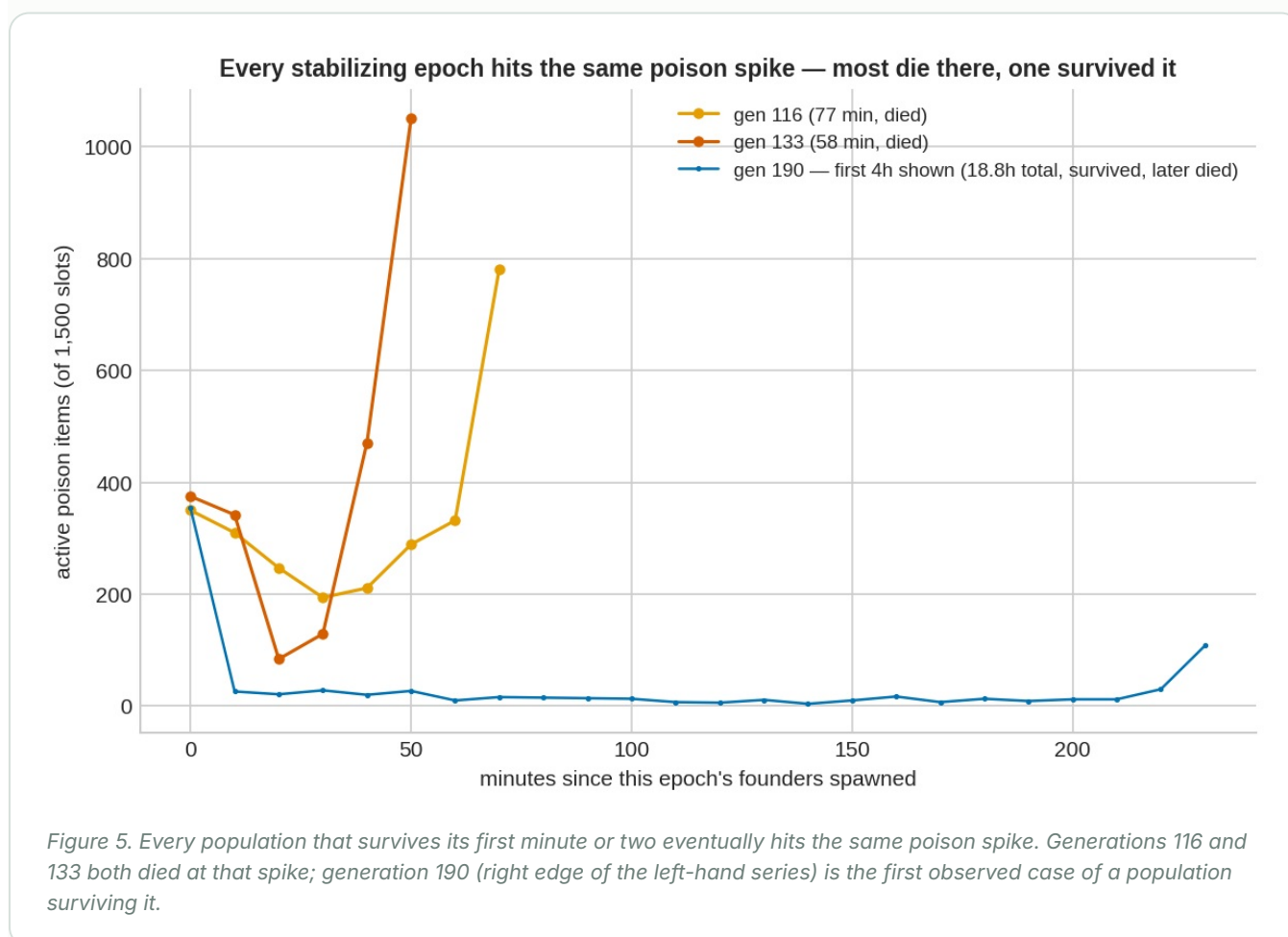
5.2.6 The extinction spree, and the second long-surviving epoch

133 extinctions occurred across the 4.8 hours immediately following the first epoch's collapse: a median lifetime of **1.3 minutes**, with 97% of epochs lasting under two minutes, and a longest outlier of 77.4 minutes. This is the same founder-bottleneck filter documented in the heavier sandbox, noticeably harsher here: essentially every fresh 100-founder population died almost immediately, with no toxin-resistance gene and no nutrient buffer to fall back on.

Extending the log by a further 25 hours revealed a fuller and more interesting picture. Two later epochs also "stabilized" for an extended period:

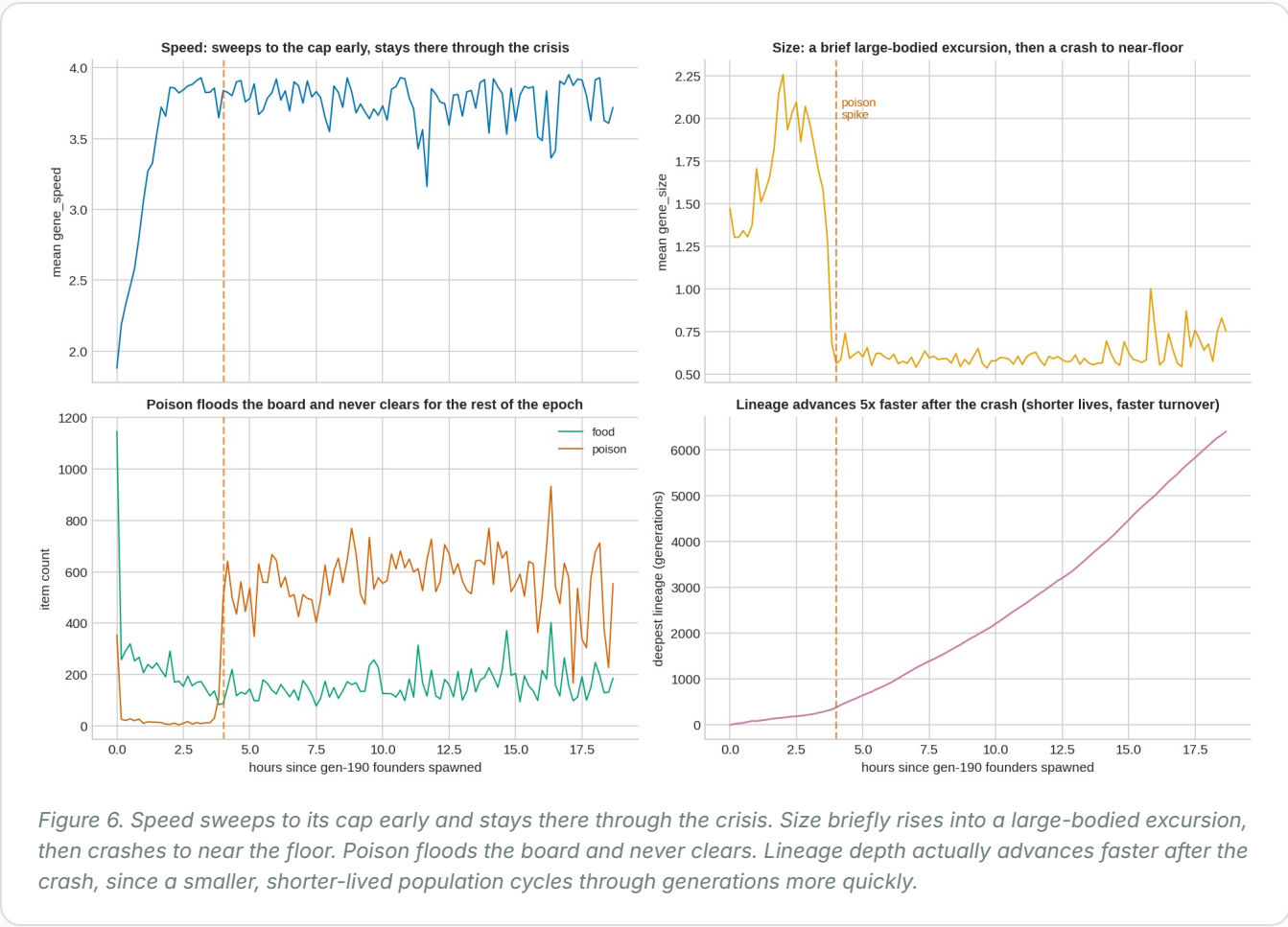
Epoch	Duration	Outcome	Final lineage depth
generation 116	77 minutes	Died	109
generation 133	58 minutes	Died (had looked promising at an earlier check-in, but did not go on to survive)	96
generation 190	18.8 hours	Died, but only after surviving an internal poisoning crisis	6,401
generation 260	3.0 hours so far	Still running as the extended log ends	245

Lining up the poison-item trajectory of the three shorter, well-documented "stabilizing" attempts (generations 116, 133, and the early part of 190) from each one's own founding moment reveals a common pattern: every population that survives its first minute or two eventually runs into the same wall, with poison items climbing steadily and then spiking sharply (to 780 and 1,050 items respectively in generations 116 and 133) right before the population collapses.



Generation 190 is the first observed case in this project of a population **surviving that spike rather than being killed by it**:

	Before the crisis (h 0–3.8)	After the crisis (h 4.2–18.7)
Mean population	79	57
Mean size	1.66	0.62
Mean poison count	33	567 (17× higher)
Mean food count	243	159
Mean maximum age	10,739	1,677 (6.4× shorter)
Lineage-advance rate	~78 generations/hour	~415 generations/hour (5.3× faster)



Population never recovers its pre-crisis size or age profile, and poison never clears, but the population does not die out either — it settles into a smaller, shorter-lived, faster-turnover regime that persists for another 15 hours, advancing the bulk of its eventual 6,401-generation lineage depth *after* the crash, purely because generations are so much quicker when individuals are small and short-lived. This re-frames the earlier "self-poisoning kills the epoch" reading from the first long epoch: self-poisoning does not have to be fatal. A population that shrinks fast enough can keep finding the shrinking pool of food faster than it dies, and survive indefinitely in a permanently polluted, low-quality environment, at a fraction of its earlier population and lifespan — a much closer parallel to the heavier sandbox's alternating size regimes than the original 16.8-hour epoch suggested, except here the small regime is a

forced adaptation to a self-inflicted crisis rather than a life-history choice. Confidence: high for the sequence of events; the underlying question of why this particular population survived its crisis while the two shorter ones did not cannot be answered from population means alone.

Across all 259 completed epochs logged over the full 46.8-hour extended run, the median lifetime remained essentially unchanged at about 1.2 minutes, and only 3 of 259 (1.2%) ever lasted more than 15 minutes — reinforcing the reading that evolutionary rescue in this world is a rare, largely luck-driven event that does not get more likely with practice, even as the ceiling on how long a successful population can eventually run (16.8 hours, then 18.8 hours) has crept upward each time one succeeds.

5.2.7 Phenomena not observed, and why

Trophic diversification, thermal adaptation, immune drift, and seasonal cycling cannot appear in this build at all — it has no chloroplast, scavenger, carnivore, insulation, immunity, or season/era system, so this is a scope limitation of the lite sandbox, not a population-level finding. Colour-based speciation and detailed predation-pressure dynamics are plausible in principle, since the mating rule needs hue similarity and predation needs a size/energy edge, but neither hue nor kill events are logged in this CSV, so neither can be directly checked.

5.3 Planetary Sandbox v3 (server_v3.js) — A Build That Never Gets Off the Ground

Data: evolution_v3_log.csv , 1,012 rows across one continuous 20.8-hour session, 2026-09-23 04:18 UTC to 2026-09-24 01:06 UTC.

5.3.1 The headline result: total reproductive lockdown

	Value
Total extinctions logged	1,010
Total runtime	20.8 hours
Epochs that ever reached a lineage depth of 2 or more	Zero, out of 1,010
Lineage-depth values ever observed	Only 0 (271 rows) or 1 (741 rows) — nothing else, at any point
Median epoch duration	61 seconds
Mean epoch duration	74 seconds
Longest epoch ever observed	325 seconds (5.4 minutes)
Trait drift over 20.8 hours	None detectable — every trait mean stays within its founders' random-draw noise band the entire time

This is a qualitatively different result from every other build examined in this report. In the heavier sandbox, the lite sandbox, and even the ESP32 hardware build (\$5.4), most founder populations die quickly, but some eventually break through and establish a deep lineage — that is exactly what "evolutionary rescue" means. In this log, across 1,010 independent tries, that never happened even once. Births do occur in large numbers each epoch (a cumulative total of 210,802 births by the end), so parents are managing to have first-generation children — but no first-generation child ever survives long enough to have a child of its own before the whole population collapses.

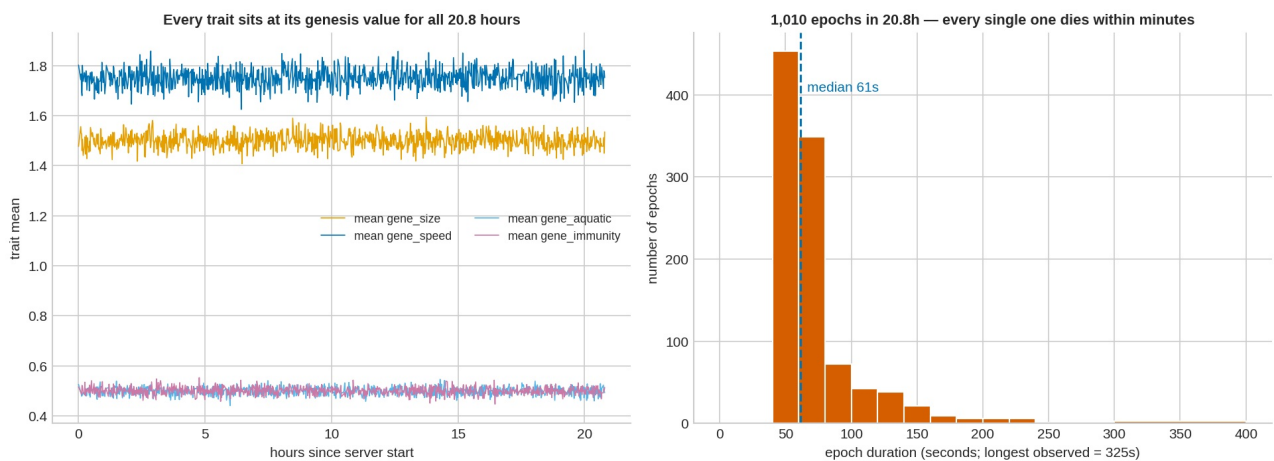


Figure 7. Left: mean size, speed, aquatic affinity, and immunity across the full 20.8-hour run, all pinned to their genesis values with only sampling noise. Right: all 1,010 epoch durations; the longest ever observed was 325 seconds.

5.3.2 The likely cause: an under-tuned terrain penalty

The strongest candidate, based on reading the code, is the newly added aquatic/terrestrial terrain mechanic. A creature mismatched with its local terrain — a non-aquatic creature standing in "water," or an aquatic creature stranded on "land" — loses 10 energy per tick, compared with an ordinary resting metabolic cost on the order of 0.04 to 0.07 energy per tick for a founder-sized creature: a mismatch costs roughly **150 to 250 times normal metabolism**. The same mismatch also cripples the creature's ability to escape, multiplying its speed by only 0.3 (non-aquatic in water) or 0.1 (aquatic on land).

Mapping the terrain function used in this build shows that **58% of the arena counts as "underwater."** Given founders spawn with their aquatic-affinity gene drawn uniformly from 0 to 1 (so roughly half are "aquatic" by the code's own greater-than-0.5 threshold) at uniformly random positions on a map that is 58% water, a large share of every founder batch is terrain-mismatched from the very first tick — and mismatched creatures are both bleeding energy fast and too slow to reliably reach terrain that matches their genes. This is consistent with the observed pattern: many first-generation offspring are born, since parents that happen to survive briefly can still reproduce, but essentially none of those offspring — who inherit a similar random spread of the aquatic gene — get the lucky combination of matched terrain and enough time to reproduce themselves. Confidence: high that this mechanic is a major contributor, based directly on the code's own numbers; the exact share of deaths it causes cannot be confirmed without per-creature death-cause logging, which this build does not record.

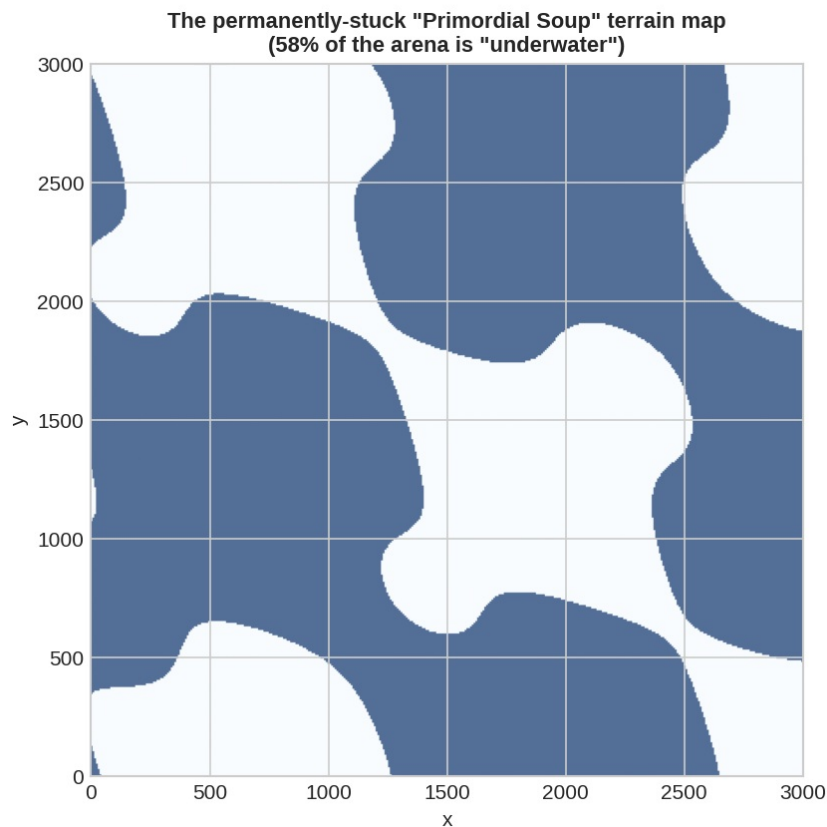


Figure 8. The elevation map this build has used for its entire 20.8-hour run (shaded regions are "underwater," 58 percent of the arena). Founders spawn at uniformly random positions with a uniformly random aquatic-affinity gene, so a large fraction land in a terrain their genes do not match, and once mismatched, cannot move fast enough to escape before the energy drain proves fatal.

5.3.3 A second finding: two of the new systems never actually run

Two of this build's headline additions never had a chance to matter in this log, for a much simpler reason — they are never advanced. The deep-time geological epoch counter is initialized to zero and is never incremented anywhere in the code, so the world has been permanently stuck in "Primordial Soup," the very first of its six eras, for the entire run — the size cap has stayed at 2.5 rather than ever reaching the Carboniferous era's 5.0, and the radiation level has stayed fixed except during a manual button press. Separately, the season counter never advances either — every one of the 1,012 logged rows shows season zero. Advancing a season requires 3,000 ticks, which at the nominal rate of 200 ticks per second is only 15 seconds, but a comment in the code itself estimates about two minutes of real time per season, implying the developer expected a real throughput closer to 25 ticks per second under load — and even at that slower rate, a median 61-second epoch only completes about half a season. This suggests the real achieved tick rate in this build is well below its nominal figure, most likely because the per-tick cost of sensing and pairwise interaction checks across up to 600 creatures and 1,500 items, plus the new per-tick terrain calculation, is heavier here than in the heavier sandbox. Confidence: moderate, since this is inferred from the season and era counters never moving rather than measured directly.

5.3.4 What this means for the rest of v3's new biology

Every other new mechanic in this build — sexual crossover, true neural kin recognition, parasitism, active camouflage, host-manipulation infections, and the Permian-style toxic-rain event — requires a population to survive long enough for those interactions to occur at any meaningful scale, and ideally to persist across multiple generations for their effects to show up in trait averages. With every epoch capped at a handful of minutes and lineage never exceeding a depth of one, none of these mechanics get a real chance to operate in this particular log. Two of them are also structurally invisible in the CSV even if they had occurred: the parasite gene is tracked per creature but is never included in the logged population averages, unlike the other trophic genes, and hue (used for active camouflage) is not logged either. This is not a criticism of the underlying design — it is a genuinely richer world on paper — but as configured and run, this 20.8-hour session could not have shown any of it.

5.4 ESP32 Hardware (`BionotesESP32.ino`) — Real Hardware, a Stable Equilibrium, No Breakthroughs

Data: `esp32_evolution_log.csv`, 1,887 rows across 21.8 continuous hours, 2026-09-23 03:16 UTC to 2026-09-24 01:06 UTC, running on real ESP32 hardware.

5.4.1 A genuine, sustained equilibrium

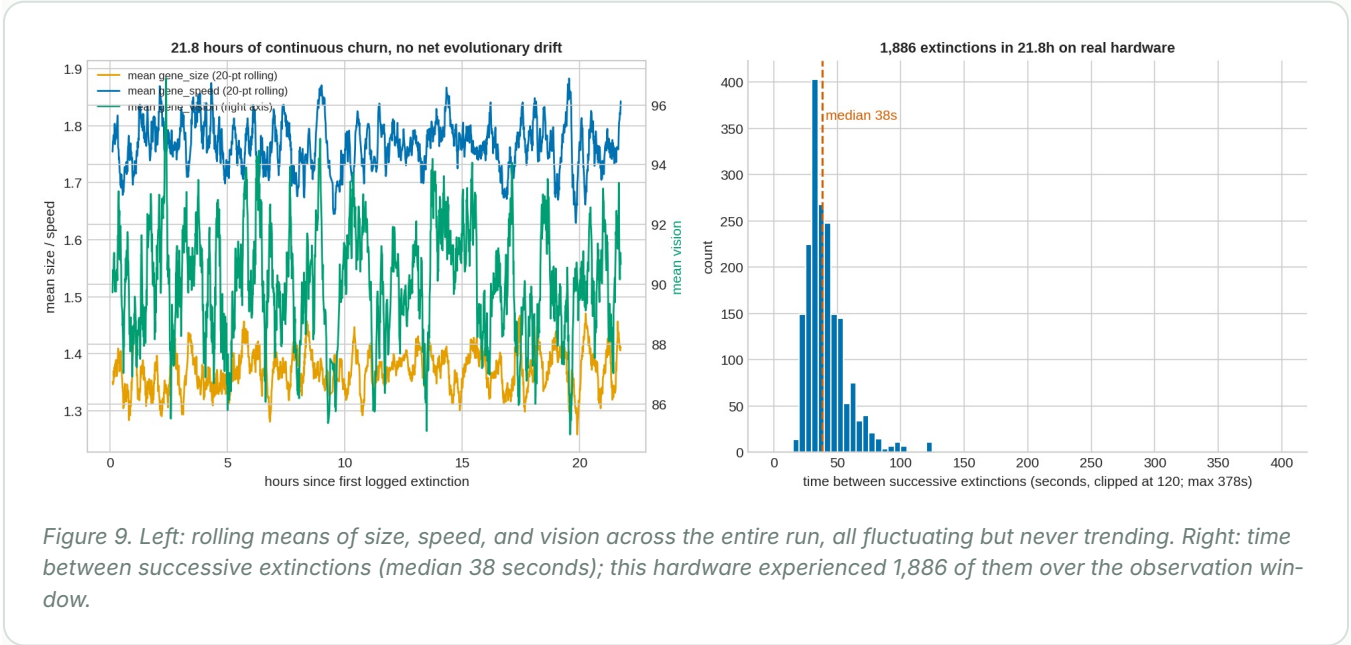
Unlike every web-based build in this report, this log shows no directional change at all over its full 21.8-hour span:

Metric	First 4 hours	Last 2 hours (of 21.8)	Change
Mean population	24.9	24.4	None
Mean size	1.358	1.386	Negligible
Mean speed	1.775	1.766	Negligible
Mean vision	90.3	90.3	None
Mean lineage depth	1.54	1.46	Negligible
Mean maximum age	17.9	14.8	Negligible

Correlating every trait mean against elapsed time across all 1,887 rows gives correlations no larger than 0.04 in magnitude for any trait — statistically indistinguishable from flat.

This is a genuinely different outcome from anything else in this report. The lite and heavier web sand-boxes each eventually produced a rare "escape" epoch that ran for many hours and showed real directional selection. Nothing like that appears anywhere in 21.8 hours of ESP32 operation — the population cycles through roughly 1,900 short-lived founder batches and consistently settles back into the same narrow trait range every time, rather than drifting toward an extreme or occasionally breaking

out. Two structural features plausibly explain why: the tiny population cap (50, versus 600 in the web sandboxes) gives genetic drift far more relative weight than selection, and the fixed-topology brain removes the one axis — neural complexity — that consistently did show unbounded, if largely inert, growth in every Node.js version. Confidence: high that the pattern (no drift, no breakout) holds in this log; the causal explanation is a reasonable inference, not something this data alone can isolate.



5.4.2 Lineage depth: shallow and capped, but not at exactly one

Deepest lineage reached	Number of epochs (of 1,887 logged)
0	85
1	1,001
2	647
3	133
4	19
5	2

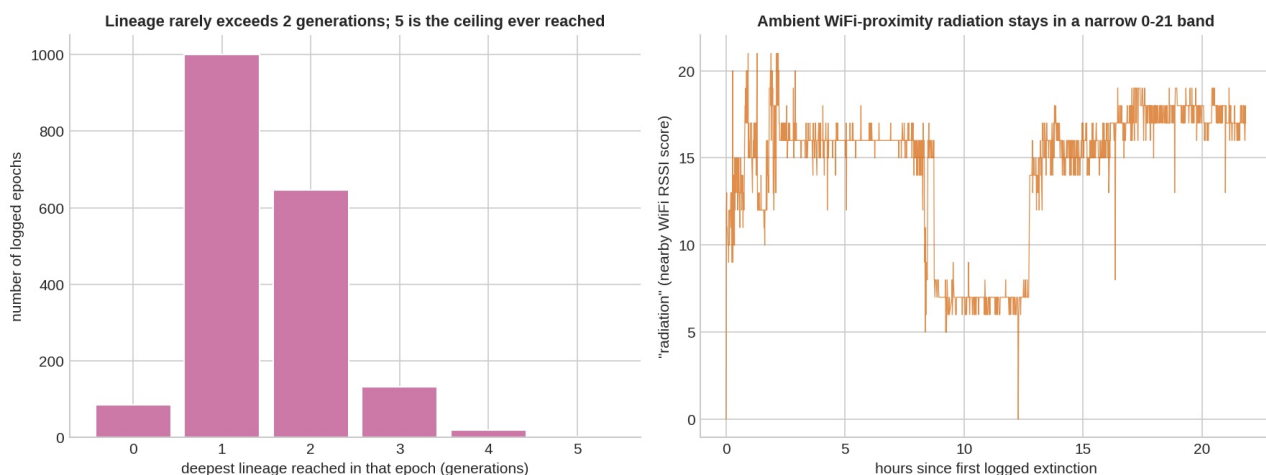


Figure 10. Left: almost every epoch tops out at one or two generations; five is the deepest ever observed. Right: the WiFi-proximity radiation signal over the full run, including a multi-hour dip to near zero around hours 8.5 to 12.5.

This sits between the two extremes elsewhere in this report: it is not the near-total lockout seen in `server_v3.js` (where lineage never exceeded 1 even once), but it falls far short of the thousands-of-generations epochs eventually reached by the lite and heavier web sandboxes. A tiny, fast-cycling population here manages an occasional second or third generation before its short-lived batch dies out, but never breaks through to sustained growth. Confidence: high.

5.4.3 The WiFi-radiation and day/night mechanics, checked against the data

Radiation stayed in a narrow, moderate band (0 to 21, mean 14.9) for the whole run, with one clear multi-hour dip to roughly 5–8 between hours 8.5 and 12.5 — plausibly a period when fewer phones or devices were physically near the chip. Outside that window, radiation mostly sat in the 14–19 range. There was no detectable effect of radiation on population, speed, or epoch duration in this particular log (all such correlations were under 0.02 in magnitude), though given how rarely radiation swung far from its typical band, this is weak evidence at best that the mechanic does not matter — a deliberate test, such as leaving a phone next to the chip for an extended stretch, would be a more direct way to check. The day/night vision mechanic cannot be checked against the logged vision column at all, since that column records the underlying vision gene, not the moment-to-moment sensing range the day cycle actually modulates on top of it.

5.4.4 A data-quality note on epoch age

The logged epoch-age field tracks almost exactly with total elapsed wall-clock time since the first logged row (78,597 seconds logged against 78,598.5 seconds of real elapsed time) rather than resetting after each extinction, even though the firmware source resets its internal epoch-start timer on every reset. In other words, this field behaves like total device uptime, not per-epoch age, for this particular log. This report reconstructed real epoch durations from the time between consecutive logged rows instead, which lines up well with the extinction counter, since that column increments by exactly one on every single row with no exceptions across all 1,886 transitions — consistent with the logger recording one row per extinction event rather than polling on a fixed timer. This is flagged

plainly as a caveat rather than silently worked around, since it means the epoch-age field is not usable as logged, whatever the underlying reason.

5.4.5 Comparison at a glance: ESP32 versus the three Node.js builds

	ESP32 hardware	<code>server.js</code> (lite)	<code>server_v2.js</code> (heavier)	<code>server_v3.js</code> (planetary)
Population cap	50	600	600	600
Brain	Fixed 8-neuron network	NEAT-style (growing topology)	NEAT-style	NEAT-style
Median epoch duration	38 seconds	~1.2 minutes	~1.5 minutes	~61 seconds
Longest epoch ever seen	~378 seconds	18.8 hours	78.5 hours (ongoing)	325 seconds
Long-term trait drift	None observed	Strong, once an epoch escapes	Strong, once an epoch escapes	None possible — no epoch ever escapes
Unique real-world input	WiFi-proximity radiation, real day/night cycle	None	None	None

The ESP32 build and `server_v3.js` both show "no long-term evolution" in their logs, but for different reasons: v3's populations die too fast and too uniformly for evolution to have any chance at all (a tuning issue, per §5.3.2), while the ESP32 build's populations survive about as long on average as v3's did, yet do occasionally reach a few generations deep — it simply never compounds into a break-out, plausibly because the population is too small and the brain too simple for a lucky lineage to snow-ball the way it did in the larger web sandboxes. It is a smaller-scale version of the same underlying story: getting past the founder bottleneck appears to be rare across every build in this project, and this is one case, over this particular 21.8-hour window, where it never happened.

5.5 Four-Way Comparative Analysis

5.5.1 At a glance

	ESP32	Lite (<code>server.js</code>)	Heavier (<code>server_v2.js</code>)	Planetary (<code>server_v3.js</code>)
Log span	21.8 hours	46.8 hours (extended)	87.9 hours, four sessions	20.8 hours
Population cap	50	600	600	600
Longest single epoch	~378 seconds	18.8 hours	78.5 hours, still ongoing	325 seconds
Founder-epoch median lifetime	38 seconds	~1.2 minutes	1.5 minutes	61 seconds
Ever reached deep, sustained lineage	No (max depth 5)	Yes (twice: 5,272 and 6,401 generations)	Yes (10,516 generations, ongoing)	No (max depth 1, ever)
Long-term trait drift	None	Strong, once escaped	Strong, once escaped	None possible
What limits success	Small population, fixed brain, drift-dominated	Self-poisoning (no nutrient recycling)	Resource limits, climate cycles	An under-tuned terrain penalty; total lockout

5.5.2 What all four builds agree on

- 1. A founder bottleneck is universal.** In every build, the overwhelming majority of fresh founder populations die within one to two minutes of wall-clock time, and only a rare epoch escapes — this falls out of the shared genesis logic (small, mostly random early brains and gene values), not a tuning artifact unique to any one version.
- 2. Where speed can reach its hard cap, it does.** In the three builds where lineage survives long enough to matter (lite, heavier, and to a lesser extent the ESP32 build), mean speed climbs toward its ceiling and stays there.
- 3. Brain wiring, where it can grow, grows roughly linearly with no plateau, and is mostly inert.** The identical single-pass evaluation bug (§3.10) appears in all three Node.js builds, confirmed with the same direct test in each.
- 4. Evolutionary rescue reads as a rare, largely stochastic event everywhere it occurs,** rather than something that becomes more likely with practice. The heavier sandbox needed 196 failed attempts before one succeeded; the lite sandbox's very first attempt succeeded, and a second, unrelated attempt later succeeded again; the ESP32 build never produced a clean success in this log at all; and the planetary sandbox never produced one either, for an entirely different reason.

5.5.3 What separates the four builds, and why

Divergence	Explanation
Heavier sandbox's population grows toward its size ceiling; lite sandbox's shrinks toward its floor	Opposite metabolic cost curves: Kleiber-style scaling (economy of scale for large bodies) in the heavier build versus a purely linear cost structure in the lite build, with no such economy
The heavier sandbox's long epoch is still running; the lite sandbox's first long epoch died of self-poisoning	The heavier build closes its nutrient loop and gives creatures a scavenger gene that can profit from corpses; the lite build has neither, so unrecycled corpses eventually crowd out food entirely
The lite sandbox's second long epoch (generation 190) survived the same kind of poisoning crisis that killed its first	A population that shrinks and speeds up its generational turnover fast enough can outrun a slowly worsening environment rather than being wiped out by it — the clearest evidence in this project that self-poisoning is a survivable, not always fatal, event
The planetary sandbox (v3) never produces a surviving population at all	An added terrain-mismatch penalty is severe enough (on the order of 150 to 250× normal metabolism, with an escape speed cut to 10–30%) that, combined with 58% of the arena being "underwater," it appears to overwhelm every other mechanic in the build before they can matter
The ESP32 hardware build reaches a stable equilibrium and never breaks out	A much smaller population cap (50 versus 600) leaves genetic drift dominant over selection, and a fixed-topology brain removes the one axis — unbounded (if largely inert) neural growth — that every Node.js build showed

5.5.4 Bottom line

All four systems share the same underlying engine logic: variation, heritability, and differential survival, wired up through a simple evolving neural network. What differs enormously is the outcome that logic produces, and in every case the difference traces back to a small number of numerical constants or a handful of lines of code — a cost curve, a nutrient-recycling loop, a terrain penalty, a population cap, a fixed versus growable brain. That sensitivity is itself one of the most useful lessons this project demonstrates: a system built from genuinely biological first principles does not automatically produce interesting, life-like evolution just because the ingredients (mutation, selection, reproduction) are present — it also has to be tuned so that survival is hard, but not so hard that nothing ever survives it, as `server_v3.js`'s total lockout demonstrates by its absence of any success at all.

6. Communicating the Project: Outreach Frameworks

The codebase translates well into public-facing demonstrations. Two ready-made formats:

A 60-second short-form video pitch — open on the glowing arena; show Generation Zero chaos (random brains, most individuals starving or hitting poison within seconds); cut to a single surviving strain multiplying around a food source by around the ten-minute mark; close on a live neural-connection visualizer with the line that the entire project runs with no cloud servers and no human supervision. Given the findings in §5.3, the planetary sandbox is not currently a good candidate for this demonstration — it's worth running the heavier or lite sandbox instead until the terrain-penalty tuning issue is addressed.

A live science-expo booth, pairing an ESP32 running the firmware at full frame rate with a laptop projecting a dashboard from the Node.js server, plus a physical button wired to a meteor-strike event that wipes out most of the population, demonstrating a genetic bottleneck live. A three-step demo sequence works well with a general audience: a "genesis reset" (watch the initial chaos and near-total die-off), an "aha" moment (open the neural visualizer on a surviving lineage and point out a working sensor-to-motor connection), and a "cosmic catastrophe" (trigger the meteor event and watch the population adapt to sudden famine).

7. Installation & Execution

7.1 ESP32 Microcontroller (`BionotesESP32.ino`)

1. Open the file in the Arduino IDE.
2. Set your local WiFi credentials in the `ssid` / `password` constants.
3. Compile and flash the sketch to an ESP32 development board.
4. Watch the Serial monitor (115200 baud) for the assigned IP address.
5. Open that IP address in a browser to view the live dashboard.

If you want to reproduce the kind of external CSV log analyzed in §5.4, poll the device's `/api/state` endpoint on a short interval and append each response as a row — be aware, per §5.4.4, that the device's own reported epoch age does not reliably reset per epoch, so reconstruct epoch durations from your own polling timestamps instead if precise timing matters.

7.2 Node.js Servers (`server.js` , `server_v2.js` , `server_v3.js`)

```
cd petriChipNode
npm install express socket.io dotenv
# optional, for the Gemini-powered evolutionary observer:
echo "GEMINI_API_KEY=your_key" > .env

node server.js      # NEAT-Lite engine (the "lite" sandbox)
# or
node server_v2.js   # closed-loop biogeochemical sandbox (the "heavier" sandbox)
# or
node server_v3.js   # planetary sandbox with terrain, parasitism, and deep-time cycles
```

Then open `http://localhost:3000` in a browser. Given the findings in §5.3, anyone running `server_v3.js` for research purposes should expect to see essentially no multi-generational evolution until the terrain-penalty tuning issue described there is addressed.

8. Known Limitations

- **Aggregate-only logging, across every build.** All four CSV logs record population *means* only — no per-individual data, distributions, or guild counts — so many of the readings in §5 (trophic specialization, punctuated equilibrium, the terrain-penalty hypothesis) are strong, code-supported hypotheses rather than direct measurements. A shifted mean could reflect a uniform population-wide change or a shift in the mix of subgroups.
- **The inert hidden-layer bug**, documented in §3.10 and confirmed by direct test in all three Node.js builds, means most logged "brain growth" across this entire project is neutral structural bloat rather than functional neural elaboration.
- **Wall-clock rates depend on load.** Simulation tick throughput appears to scale with population and item count, so hourly rates (connections/hour, births/minute) are approximate, not fixed per-tick rates — and, in `server_v3.js`, the season and geological-era counters never advancing at all suggests the real achieved tick rate can fall well below the nominal figure under sustained load.
- **Unlogged interventions.** Manual events (radiation bursts, meteor strikes, food drops) are not flagged in any of the CSV logs, so some sudden shifts in the data cannot be cleanly separated from ordinary evolutionary dynamics.
- **Several v3-specific genes and traits are not logged at all.** The parasite gene and creature hue are both tracked internally in `server_v3.js` but never appear in its CSV output, so parasitism prevalence and colour-based camouflage cannot be assessed from the log even in principle, independent of whether populations survive long enough for them to matter.
- **The ESP32 build's logged epoch-age field is not usable as logged** (§5.4.4); all ESP32 epoch-duration figures in this report were reconstructed from inter-row timestamps instead.
- **Season/era aliasing.** Ten-minute sampling intervals can alias with faster season cycles, especially early in a run, in both `server_v2.js` and `server_v3.js`.

8.1 Suggested Logging Improvements

- Per-gene standard deviation or histograms, to detect real fixation or bimodality rather than only mean shifts.
- Explicit counts per trophic class, rather than population means alone.
- A logged era and geological-epoch value, plus an actual ticks-per-second figure, to normalize rates and confirm or refute the tick-rate concerns raised in §5.3.3.
- Event flags for meteors, radiation bursts, and manual food drops.
- Separate functional vs. total connection counts per brain, so the §3.10 bug's practical impact can be measured directly rather than inferred.
- A predation-event counter, independent of the general death counter.

- Mean hue and hue variance, to test for colour-based assortative mating and speciation.
 - A per-creature death-cause tally (starvation, predation, senescence, viral, terrain mismatch), which would directly test the §5.3.2 terrain-penalty hypothesis instead of leaving it inferred from code alone.
 - For the ESP32 build specifically, a properly per-epoch-resetting age field, and a direct measurement of the effective (day/night-modulated) sensing range alongside the underlying vision gene.
-

9. Project Synthesis

The central claim of `petriChip` is that life and artificial intelligence share the same underlying architecture: given a medium governed by thermodynamics, localized senses, and mutable neural connections, biological principles can emerge spontaneously from a few hundred lines of code. With full access to the codebase and to every available telemetry log, that claim holds up well — but with an important qualification that this second edition can now make explicit, where the first could not.

Two of the four builds examined here — the heavier and lite Node.js sandboxes — do produce exactly the rich, emergent story the project sets out to tell: founder bottlenecks, evolutionary rescue, detritivorous recycling, punctuated body-size equilibria, directional selection running into hard genetic ceilings, and even a population that learned, in effect, to survive its own pollution by shrinking and speeding up rather than by cleaning up after itself. A namable software bug — functionally inert hidden-layer neurons — reframes what "growing brain complexity" means throughout every one of these logs, without diminishing the genuine evolutionary dynamics happening around it.

But the other two builds tell an equally important, opposite kind of story. The newest planetary sandbox, `server_v3.js`, adds substantial new biology on paper — deep geological time, dynamic terrain, sexual crossover, parasitism, active camouflage — and then, in the 20.8 hours of continuous operation examined here, never once lets any of it happen: an under-tuned terrain penalty appears to close the door on evolution before it can start, and two of its other new systems turn out to be permanently stuck rather than merely slow. The real ESP32 hardware build tells a subtler version of the same lesson: it is not broken in any comparable way, but its smaller scale and simpler, fixed brain settle it into a stable equilibrium that never produces the kind of long-run breakthrough its larger siblings eventually did.

Whether run in the flash memory of a five-dollar microcontroller or scaled across a multi-thousand-entity Node.js server, the same core dynamic holds: given variation, heritability, and differential survival, a purely computational system can rediscover many of the same adaptive patterns that carbon-based life arrived at over billions of years. But this project's fuller record now also shows the other side of that coin just as clearly: the same system, with the same underlying logic, can just as easily produce total reproductive lockout or a directionless equilibrium if the balance of survival pressure is even slightly wrong — and figuring out exactly where the tuning went wrong, using nothing but code and telemetry, turns out to be its own small exercise in evolutionary biology.

Compiled from the project's source code (`BionotesESP32.ino`, `server.js`, `server_v2.js`, `server_v3.js`) and from full statistical analysis of the raw CSV telemetry from all four builds (`esp32_evolution_log.csv`, `evolution_log.csv`, `evolution_v2_log.csv`, `evolution_v3_log.csv`), available at github.com/titasmallick/petriChip.